F24-047-D-ScholarChain

AI-Driven Student Loaning Platform on Blockchain

Project Team

Tayyaab Ali Sajid	21I-2546
Muhammad Riyan Aslam	21I-0428
Saifullah Rizwan	21I-0830

Session 2021-2025

Supervised by

Dr. Muhammad Asim



Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

1 Introduction **1**

1.1 Existing Solutions . . . 1
1.2 Problem Statement . . . 1
1.3 Scope . . . 2
1.4 Modules . . . 2
1.4.1 Module 1 - AI Module . . . 3
1.4.2 Module 2 - Blockchain Module . . . 3
1.4.3 Module 3 - Web Application . . . 3
1.5 Work Division . . . 4

2 Project Requirements **5**

2.1 Use-case . . . 5
2.1.1 Use Case: Apply for Loan . . . 5
2.1.2 Use Case: Provide Funding . . . 6
2.1.3 Use Case: Approve Loan . . . 7
2.1.4 Use Case: Track Repayment . . . 8
2.1.5 Use Case: Customer Support Interaction . . . 9
2.1.6 Use Case: Personalized Repayment Plan Generation . . . 11
2.1.7 Use Case: Automated Loan Disbursement . . . 11
2.1.8 Use Case: Verify Loan Agreement . . . 12
2.1.9 Use Case: Payment Scheduling and Notification . . . 13
2.2 Use Case Diagram . . . 15
2.3 Functional Requirements . . . 16
2.3.1 Module 1: Student Interaction Module . . . 16
2.3.2 Module 2: Funding and Loan Management Module . . . 16
2.3.3 Module 3: Administrative Tools . . . 16
2.3.4 Module 4: Notifications and Reminders . . . 17
2.4 Non-Functional Requirements . . . 17
2.4.1 Reliability . . . 17
2.4.2 Usability . . . 17
2.4.3 Performance . . . 18
2.4.4 Security . . . 18

3 System Overview **19**
3.1 Architectural Design . . . 19
3.1.1 Box and Line Diagram . . . 19
3.1.2 Component Diagram . . . 20
3.2 Data Design . . . 21
3.2.1 Json Schema . . . 21
3.3 Domain Model . . . 30
3.4 Design Models . . . 31
3.4.1 Activity Diagram . . . 31
3.4.2 Class Diagram . . . 32
3.4.3 Sequence Diagrams . . . 33
3.4.3.1 Loan Application Sequence Diagram . . . 33
3.4.3.2 Funding Contribution Sequence Diagram . . . 33
3.4.3.3 Approve Loan Sequence Diagram . . . 34
3.4.3.4 Track Repayment Sequence Diagram . . . 34
3.4.3.5 Customer Support Sequence Diagram . . . 35
3.4.3.6 Payment Scheduling and Notification Sequence Diagram 35
3.4.3.7 Personalized Repayment Sequence Diagram . . . 36
3.4.3.8 Automated Loan Disbursement Sequence Diagram . . . 36
3.4.3.9 Verify Loan Sequence Diagram . . . 37
3.4.4 State Transition Diagram . . . 38

4 Implementation and Testing **39**
4.1 Algorithm Design . . . 39
4.2 External APIs/SDKs . . . 44
4.3 Testing Details . . . 44
4.3.1 Unit Testing . . . 44

5 Conclusions and Future Work **47**
5.1 Conclusion . . . 47
5.2 Future Work . . . 48

References **49**

A Appendices **51**
A.1 Appendix A . . . 51
A.1.1 Use Case Diagram example (Online Shopping System) . . . 51
A.1.2 Detail Use Case Example . . . 52
A.1.3 Event-Response Table for a Highway Intersection . . . 53
A.1.4 Story Board Example For Android App . . . 54
A.2 Appendix B . . . 55

A.2.1 Domain Model Example For Online Shopping 55
A.3 Appendix C 56
A.3.1 Box And Line Example For Online Shopping 56
A.3.2 Architecture Pattern Example For Online Shopping 57
A.4 Appendix D 58
A.4.1 Activity Diagram 58
A.4.2 Class Diagram 59
A.4.3 Sequence Diagram 60
A.4.4 State Transition Diagram 60
A.4.5 Data Flow Diagram 62

List of Figures

2.1 ScholarChain Use Case Diagram 15

3.1 Box and Line Diagram 19
3.2 component Diagram 20
3.3 ScholarChain Domain Model 30
3.4 activity Diagram 31
3.5 Class Diagram 32
3.6 Loan Application Sequence Diagram 33
3.7 Funding Contribution Sequence Diagram 33
3.8 Approve Loan Sequence Diagram 34
3.9 Track Repayment Sequence Diagram 34
3.10 Customer Support Sequence Diagram 35
3.11 Payment Scheduling and Notification Sequence Diagram 35
3.12 Personalized Repayment Sequence Diagram 36
3.13 Automated Loan Disbursement Sequence Diagram 36
3.14 Verify Loan Sequence Diagram 37
3.15 State Transition Diagram 38

A.1 Use Case Diagram for the Online Shopping System 51
A.2 Detail Use Case Example 52
A.3 Example of Event Response Table 53
A.4 Example of Story Board 54
A.5 Domain Model Example For Online Shopping Application 55
A.6 Box and Line Diagram For Online Shopping Application 56
A.7 Architecture Pattern For Online Shopping Application 57
A.8 Activity Diagram For Online Shopping Application 58
A.9 Class Diagram For Online Shopping Application 59
A.10 Sequence Diagram For Online Shopping Application 60
A.11 State Transition Diagram For Online Shopping Application 60

List of Tables

1.1 Work Division of project Modules 4

4.1 List of APIs/SDKs used in the project 44
4.2 Unit Test Cases for Loan Applications Part 1 45
4.3 Unit Test Cases for Loan Applications Part 2 46

Chapter 1

Introduction

The following project proposal focuses on ScholarChain, which is an AI-driven Student Loan System based on the Blockchain that aims to provide interest-free loans to students to help them pursue their educational ambitions. ScholarChain revolves around revolving funds that are crowd-sourced via social welfare and charitable organizations, allowing such organizations to have a transparent and secure platform to connect with those who need their services. The proposal aims to provide a detailed description of the system and its core features, as well as the motivation behind the project.

1.1 Existing Solutions

EduFi is a student loaning platform that provides loans to students so that they may pursue their higher education with relative ease due to their lenient repayment plans. Through its friendly online portal, EduFu aims to connect students with financial resources to help cover tuition and other educational expenses, offering flexible repayment schedules. However, the main concern with EduFi's model is the absurdly large processing fee. This is a one-time charge applied to the loan amount. While not categorized as interest, this fee approaches or even exceeds the legal limit on interest rates set by Pakistan's State Bank.

1.2 Problem Statement

Quality education is a fundamental human right. However, this human right is slipping out of the reach of a lot of people. This is due to the extremely high tuition fee that is not affordable by most students. This continuous rise in costs is causing students to give up on their academic aspirations entirely, either by dropping out halfway or choosing not to pursue a higher education at all. While financial support systems do exist, they are inad-

equate and often come with their own unique set of problems that can become a burden on the students in their own way. Loans are accompanied by strict repayment schedules and high interest rates which can lead to long-term debts. According to [**?**], the application process for such financial aid services is also very complex which can deter many students and have stringent eligibility criteria that are not very easily met. There is also a major lack of transparency in existing systems which causes distrust amongst the shareholders. Furthermore, charitable organizations face difficulties in effectively distributing funds due to administrative inefficiencies. These issues contribute to turning what should be an effective solution to the escalating costs of quality education into hurdles for the students who need it the most.

1.3 Scope

The purpose of this project is to develop an automated study loaning system that will provide interest-free loans to support students with financially weak backgrounds by connecting them with social welfare organizations through a crowd-sourcing model. Through blockchain technology, the system will guarantee a transparent, secure, and impenetrable loaning process. The system will have its private blockchain to ensure the security of the transactions and transparency only within the system. Furthermore, Blockchain will be used to create chain codes that will automate the entire loan process from agreement to last loan installments and track each transaction. The system will also have a Large Language Model (LLM) which will be used to provide personal assistance to the user during the form-filling process to answer queries. Additionally, LLM will majorly be used in risk assessment of the users' application through analysis of financial documents, and academic records. LLM will also be utilized for personalized plans for loan reimbursement depending on the student's financial conditions. The system will also provide an interface for the donors to donate directly to our system without the involvement of any third parties. Moreover, students will be able to track how much loan has been reimbursed, see transaction logs, and query the chatbot in case of any confusion at any time. Lastly, the system will also automate user reminders about their repayments and automatically update the plans based on the users' timely or late payments.

1.4 Modules

Our project is divided into 3 modules. The modules are listed below.

1.4.1 Module 1 - AI Module

We will be creating a RAG (Retrieval Augmented Generation) which will enhance the user experience by utilizing the power of LLMs along with the context of the documents uploaded by the users. This Module is crucial for the success of the ScholarChain because our system will utilize the analytical power of LLM for the automation of the loaning process This module includes the following features based on AI:

1. Risk Assessment through analysis of financial documents, and academic records and checking if the applicant meets the loan criteria or not.
2. Personal Assistants such as RAGbot or field suggestions/completion to assist the users throughout the form-filling process and reduce errors.
3. Personalized loan repayment plans based on financial conditions to help the student eliminate any financial burden.
4. AI-driven customer support to query questions in case of any confusion to enhance user experience.

1.4.2 Module 2 - Blockchain Module

The blockchain module ensures that the loaning process is automated using Smart Contracts. The implementation of a ledger using the blockchain would provide security and transparency within the system to ensure a tamper-free loaning platform. The following features will be developed under the blockchain module.

1. Immutable Blockchain ledger to log all the transactions to provide a secure and transparent system.
2. chain codes to automate and enforce loan agreements, govern loan transactions, and real-time disbursements and repayment collection.

1.4.3 Module 3 - Web Application

A web application will provide an interface to the students so they can interact with the system, fill out application forms, and track their loan status. It will allow the user for real-time tracking of the loan for the students. An interface for the donors will be made to donate directly into our system using payment gateways. Additionally, the web application will also provide several insights. This module includes the following features:

1. Audit Trails from the donation till loan reimbursement to ensure complete track that the donations are being used for the proposed purpose.
2. Automated financial reports so that students can get insights into the loan status.
3. Automated reminders / notifications for upcoming or delayed payments to ensure smooth operations.

1.5 Work Division

name	**registration**	**Responsibility/Module/Feature**
Muhammad Riyan Aslam	21i-0428	(Module 1 - Feature 1, 2, 3, 4) RAG and AI related Tasks (Module 3 - Feature 3) Web Application and Automation Quality Assurance
Tayyaab Ali Sajid	21i-2546	(Module 2 - Feature 1, 2) chain codes and Ledger (Module 1 - Feature 2, 4) Personalized Assistance and Chatbot Quality Assurance
Saifullah Rizwan	21i-0830	(Module 2 - Feature 1, 2) chain codes and Ledger (Module 3 - Feature 3) Web Application and Automation Quality Assurance

Table 1.1: Work Division of project Modules

Chapter 2

Project Requirements

This section outlines the necessary requirements for the successful completion of the project.

2.1 Use-case

Fully-Dressed Use Cases

2.1.1 Use Case: Apply for Loan

Use Case	Apply for Loan
Scope	ScholarChain
Level	User Level
Primary Actor(s):	Student
Stakeholders and Interests	Student: The Student wants to apply for an interest-free loan.
Preconditions	1. The student must be registered on the ScholarChain platform and must be logged in to start the application process. 2. The students must have all required documents prepared and ready for uploading.

<table>
<tr><td>Success Guarantee</td><td colspan="2">1. Upon successful completion, the student's loan application is submitted.
2. The student receives confirmation of submission.</td></tr>
<tr><td rowspan="6">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>Student logs into the ScholarChain platform.</td><td>System authenticates the student and provides access to their dashboard.</td></tr>
<tr><td>Student selects the "Apply for Loan" option from the dashboard.</td><td>System displays the loan application form with sections for personal, academic, and financial details.</td></tr>
<tr><td>Student fills in their personal information, academic details, and financial status.</td><td>System verifies the entered details for completeness.</td></tr>
<tr><td>Student uploads the required documents.</td><td>System checks that the documents are in the correct format (e.g., PDF) and stores them securely.</td></tr>
<tr><td>Student reviews the form and submits the application.</td><td>System saves the application and sends a confirmation notification to the student.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Invalid Login
2. Incomplete Form
3. Missing Documents</td></tr>
</table>

2.1.2 Use Case: Provide Funding

Use Case	Provide Funding
Scope	ScholarChain
Level	User Level

<table>
<tr><td>Primary Actor(s):</td><td colspan="2">Social Welfare Entity (Donor)</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Donor: Wants an easy-to-use and transparent to contribute to the education of students.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The donor must be registered and logged in to the ScholarChain platform.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. Upon successful transaction, the donation is added to the immutable blockchain ledger.</td></tr>
<tr><td rowspan="6">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>Donor logs into the ScholarChain platform.</td><td>System authenticates the donor and provides access to their dashboard.</td></tr>
<tr><td>Student selects the "Contribute Funds" option from the dashboard.</td><td>The system displays funding options</td></tr>
<tr><td>Donor enters the desired contribution amount.</td><td></td></tr>
<tr><td>Donor provides payment details and confirms the contribution.</td><td>The system processes the payment securely and confirms the transaction.</td></tr>
<tr><td>Donor receives confirmation of successful payment.</td><td>The system updates the funding pool, records the transaction in the blockchain ledger, and displays the current status of contributions.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Payment Failure
2. Incorrect Contribution Amount</td></tr>
</table>

2.1.3 Use Case: Approve Loan

<table>
<tr><td>Use Case</td><td colspan="2">Approve Loan</td></tr>
<tr><td>Scope</td><td colspan="2">ScholarChain</td></tr>
<tr><td>Level</td><td colspan="2">User Level</td></tr>
<tr><td>Primary Actor(s):</td><td colspan="2">System Administrator</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants their loan application to be approved as soon as possible.
System Administrator: Needs to ensure that all documentation is in order and whether the application is deserving of loan before giving approval.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. All required documents must be successfully uploaded and verified.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. The loan application is approved, and the student is notified of the disbursement schedule.</td></tr>
<tr><td rowspan="4">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>The AI does risk assessment on the student’s application.</td><td>The system stores the results of the risk assessment to be reviewed by the system administrator.</td></tr>
<tr><td>The system administrator reviews the risk assessment results.</td><td></td></tr>
<tr><td>The system administrator approves the loan application.</td><td>The system updates the loan status to "Approved" and notifies the student.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Missing Verification</td></tr>
</table>

2.1.4 Use Case: Track Repayment

Use Case	Track Repayment
Scope	ScholarChain

<table>
<tr><td>Level</td><td colspan="2">User Level</td></tr>
<tr><td>Primary Actor(s):</td><td colspan="2">Student, Parent/Guardian</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants to ensure timely repayments and track the repayment schedule in order to manage finances accordingly.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The student or parent/guardian must be logged in to ScholarChain.
2. The loan must be approved and disbursed in order to track repayments.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. The students are able to track their repayment schedule and receive reminders about upcoming repayments.</td></tr>
<tr><td rowspan="3">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>Student logs into ScholarChain.</td><td>The system authenticates the user and displays their dashboard.</td></tr>
<tr><td>User navigates to the "Repayment Tracking" section.</td><td>The system displays detailed repayment information, including due dates, amounts, and history.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Missed payment
2. Partial payment
3. Payment processing failure</td></tr>
</table>

2.1.5 Use Case: Customer Support Interaction

Use Case	Customer Support Interaction
Scope	ScholarChain
Level	User Level

<table>
<tr><td>Primary Actor(s):</td><td colspan="2">Student, System Administrator</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants to be able to receive help with any problems related to the platform, and be able to request human support in case of a complex problem for AI.
System Administrator: Needs to assist students with issues the AI cannot resolve.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The student must be logged in to the ScholarChain platform to recieve support.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. The student's question or issue is resolved, either by the AI assistant or with manual intervention by a system administrator.</td></tr>
<tr><td>Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td></td><td>Student requests support.</td><td>The system displays available options, including an AI-powered chatbot.</td></tr>
<tr><td></td><td>Student asks a question via the chatbot.</td><td>The chatbot analyzes the question and provides a response based on available information.</td></tr>
<tr><td></td><td>Student is unsatisfied with the response or the issue is complex.</td><td>The system escalates the request to a system administrator for manual assistance.</td></tr>
<tr><td></td><td>System administrator reviews the request and responds.</td><td>The system notifies the student and displays the response from the administrator.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. No Available Administrator
2. AI inference error</td></tr>
</table>

2.1.6 Use Case: Personalized Repayment Plan Generation

<table>
<tr><td>Use Case</td><td colspan="2">Personalized Repayment Plan Generation</td></tr>
<tr><td>Scope</td><td colspan="2">ScholarChain</td></tr>
<tr><td>Level</td><td colspan="2">User Level</td></tr>
<tr><td>Primary Actor(s):</td><td colspan="2">Student</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants a flexible repayment schedule based on their current financial needs.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The student must have an approved loan.
2. The student must have submitted all relevant documentation and details</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. The system generates a personalized repayment plan, and the student is notified with the details.</td></tr>
<tr><td rowspan="4">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td></td><td>AI generates a repayment plan with due dates and amounts that are affordable for the student.</td></tr>
<tr><td>Student reviews and accepts the repayment plan.</td><td>The system records the acceptance and sets up reminders for payment schedules.</td></tr>
<tr><td>Student is unsatisfied with the response or the issue is complex.</td><td>The system escalates the request to a system administrator for manual assistance.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Outdated or missing documents</td></tr>
</table>

2.1.7 Use Case: Automated Loan Disbursement

<table>
<tr><td>Use Case</td><td colspan="2">Automated Loan Disbursement</td></tr>
<tr><td>Scope</td><td colspan="2">ScholarChain</td></tr>
<tr><td>Level</td><td colspan="2">User Level</td></tr>
<tr><td>Primary Actor(s):</td><td colspan="2">System Administrator, Blockchain Ledger</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants to receive loans as soon as possible after loan approval.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The loan must be fully approved, and the student's agreement must be signed.
2. All requirements for smart contracts must be met.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. Loan funds are successfully disbursed to the student's designated account.</td></tr>
<tr><td rowspan="4">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>The smart contract validates the loan approval conditions.</td><td>The system verifies all criteria are met for disbursement.</td></tr>
<tr><td>Upon successful validation, funds are transferred to the student's account.</td><td>The system updates the blockchain ledger with the transaction details.</td></tr>
<tr><td>Student is notified of the disbursement.</td><td>The system sends a notification with details on the amount disbursed and the repayment schedule.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Smart Contract Failure</td></tr>
</table>

2.1.8 Use Case: Verify Loan Agreement

Use Case	Verify Loan Agreement
Scope	ScholarChain
Level	User Level

<table>
<tr><td>Primary Actor(s):</td><td colspan="2">Student, Parent/Guardian</td></tr>
<tr><td>Stakeholders and Interests</td><td colspan="2">Student: Wants to be aware of the terms and conditions of the loan agreement before signing it.
Parent/Guardian: Wants to ensure that the loan agreement is fair and that all obligations are clearly stated.</td></tr>
<tr><td>Preconditions</td><td colspan="2">1. The loan must be approved and ready for signing.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. The loan agreement is reviewed and signed digitally by the student.</td></tr>
<tr><td rowspan="4">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>The student receives a notification that the loan agreement is ready for review.</td><td>The system displays the loan agreement document.</td></tr>
<tr><td>Student reviews the loan terms and conditions.</td><td>The system provides options to accept or request clarification.</td></tr>
<tr><td>student digitally signs the loan agreement.</td><td>The system records the signatures and marks the agreement as complete.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Request for Changes
2. Signature Declined</td></tr>
</table>

2.1.9 Use Case: Payment Scheduling and Notification

Use Case	Payment Scheduling and Notification
Scope	ScholarChain
Level	User Level
Primary Actor(s):	Student
Stakeholders and Interests	Student: wants to be aware of all upcoming payments.

<table>
<tr><td>Preconditions</td><td colspan="2">1. The repayment plan must already be generated and accepted by the student.</td></tr>
<tr><td>Success Guarantee</td><td colspan="2">1. Students receive notifications regarding upcoming payments.</td></tr>
<tr><td rowspan="3">Main Success Scenario</td><td>Actor Action</td><td>System Response</td></tr>
<tr><td>Student receives an alert about an upcoming payment.</td><td>The system provides details on the payment due date, amount, and instructions on how to make the payment.</td></tr>
<tr><td>Student decides to make the payment before the due date.</td><td>The system processes the payment and updates the repayment history accordingly.</td></tr>
<tr><td>Alternative Scenarios</td><td colspan="2">1. Missed Notification
2. Payment Postponement Request</td></tr>
</table>

2.2 Use Case Diagram

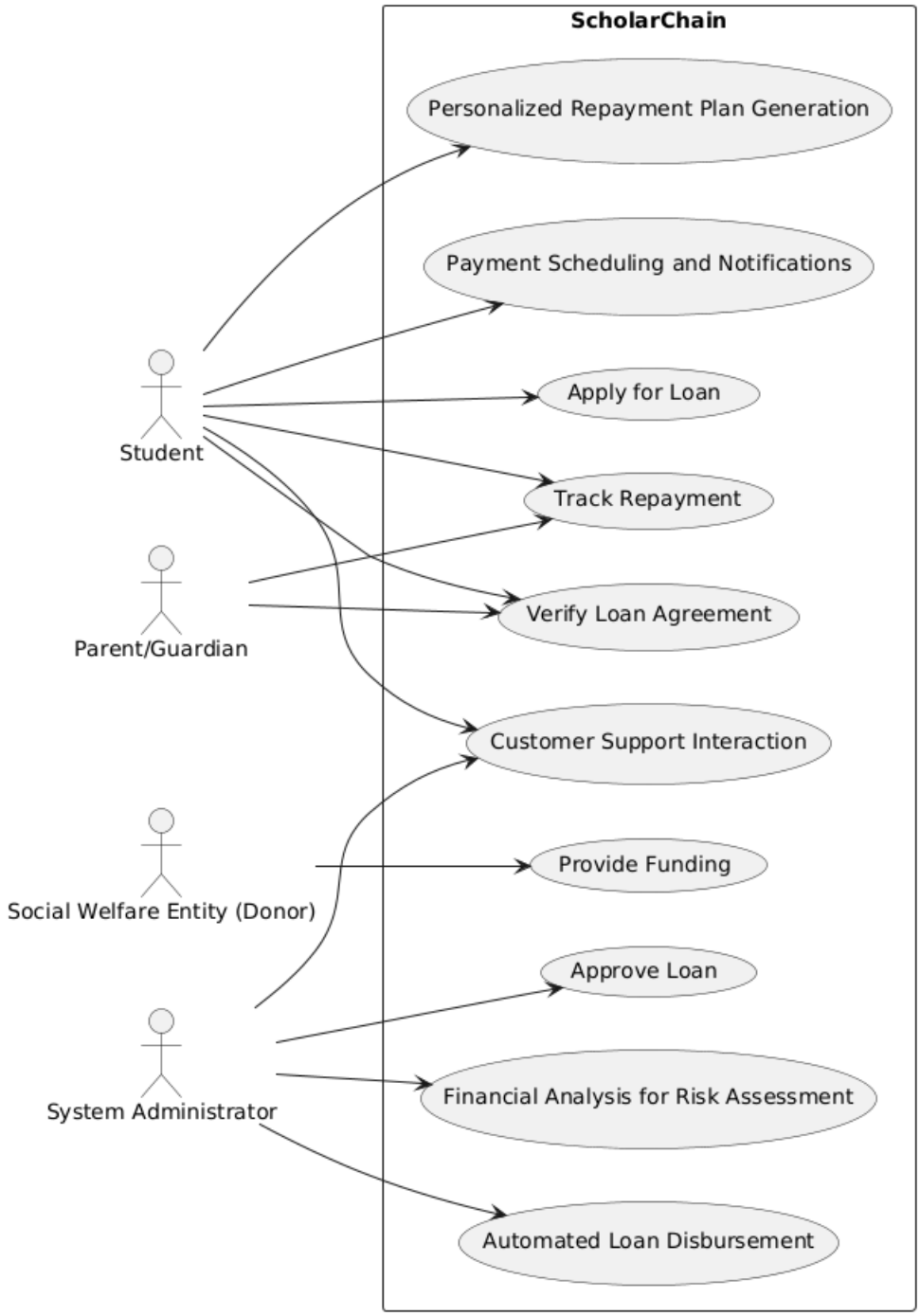


Figure 2.1: ScholarChain Use Case Diagram

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style.

2.3.1 Module 1: Student Interaction Module

The following are the requirements for Module 1:

1. **Loan Application Submission:** Students can submit loan applications by providing personal, acedemic, and financial information.
2. **Personalized Repayment Plan Schedule:** System assesses the current financial situation of the user and generates a personalized repayment plan.
3. **Repayment Tracking:** Students can track their upcoming payments and view payment history.
4. **AI-Powered Customer Support:** Students can interact with an AI-powered chatbot in order to address their queries regarding their application and the loan process.

2.3.2 Module 2: Funding and Loan Management Module

Following are the requirements for module 2:

1. **Funds Provision by Donors:** Donors can contribute funds to the platform via secure payment gateways.
2. **Loan Application Approval:** The system administrator reviews the AI-assessed application and makes the final decision on whether to approve or decline.
3. **Automated Loan Disbursement:** Funds are automatically disbursed via smart contracts one applications are approved.

2.3.3 Module 3: Administrative Tools

Following are the requirements for module 3:

1. **Automated Risk Assessment:** AI evaluates each application and assigns them a loan risk score.

2. **Verification of Loan Signature:** System administrators verify that an application's terms and conditions have been accepted by the student and has digitally signed the agreement.

2.3.4 Module 4: Notifications and Reminders

1. **Payment Scheduling and Alerts:** Automatic notifications are sent to students to remind them of upcoming payment deadlines.

2. **Loan Status Update:** The system notifies students about the current status of their loan application.

2.4 Non-Functional Requirements

This section outlines the non-functional requirements for the ScholarChain platform.

2.4.1 Reliability

ScholarChain must be highly reliable. The target mean time between failures should be no less than 500 hours. This will ensure continuous availability for the students and donors who wish to use the platform. In case of failure, automatic error detection mechanisms should be in place, as well as data protection measures to ensure that the user's data is secure. A detailed failure log should be maintained for debugging and future improvement.

- **Failure Tolerance:** The system should not experience more than one failure per month. The recovery period should not be more than 1 hour.

2.4.2 Usability

The ScholarChain platform must be easy to use for all types of users (Students, Parents, Donors). The interface must be intuitive and easy to navigate without much hassle.

- **Ease of Learning:** A new user should be able to easily understand and navigate the platform within a few minutes of interaction.
- **Error Prevention and Recovery:** Common errors such as incomplete form and uploading files in incorrect format should be clearly highlighted.
- **Interaction Efficiency:** All tasks such as submitting an application should be completable withing 5 steps.

2.4.3 Performance

The ScholarChain system should perform efficiently to provide a smooth user experience under different load conditions.

- **Response Time:** 90% of all pages must be loaded completely within 3 seconds over a 20Mbps connection.
- **Scalability:** The platform must be able to support almost 5000 concurrent users without degradation in performance.
- **Data Processing:** The data for the application process should be processable by the AI within 30 seconds of submission.

2.4.4 Security

The ScholarChain platform must be highly secure to protect user data and maintain the integrity of the system.

- **Data Protection:** All user data must be encrypted at all times.
- **Access Control:** Different users must have different access rights ensuring unauthorized users cannot access sensitive information and functionalities.

Chapter 3

System Overview

3.1 Architectural Design

3.1.1 Box and Line Diagram

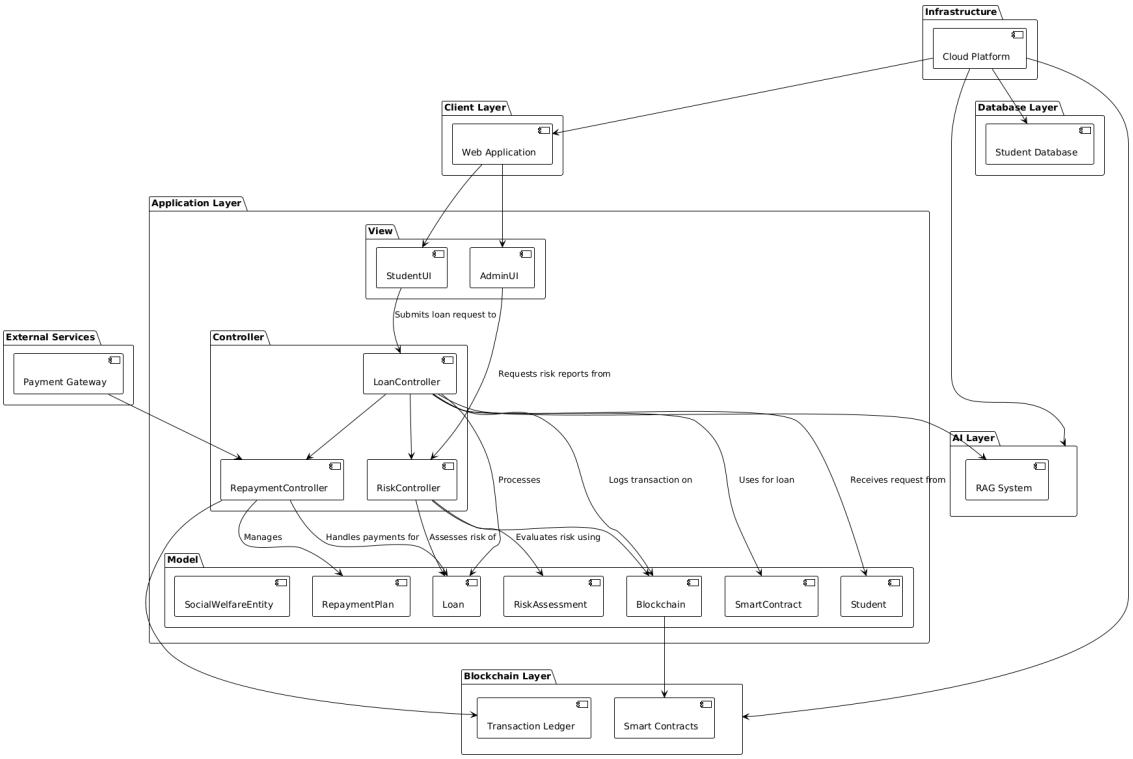


Figure 3.1: Box and Line Diagram

The box-and-line diagram is a high-level architectural representation of ScholarChain, showing how system layers interact, such as the **Client Layer**, **Application Layer**, **Database Layer**, **AI layer**, and **Blockchain Layer**.

3.1.2 Component Diagram

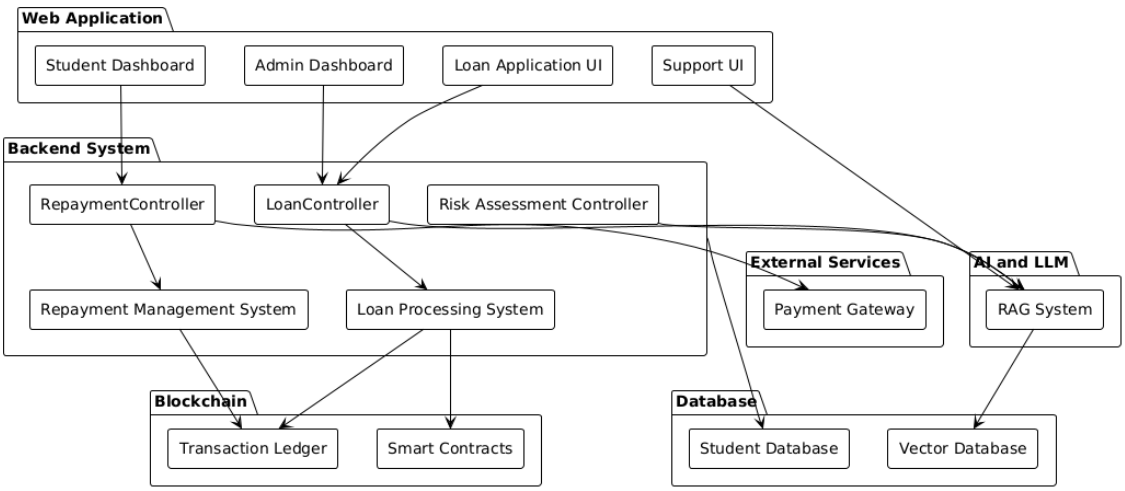


Figure 3.2: component Diagram

The Component Diagram shows a detailed architecture of ScholarChain. This diagram provides a detailed understanding of how each subsystem contributes to the functionality of the system. Connections between the different components and layers are highlighted in this diagram.

3.2 Data Design

3.2.1 Json Schema

Application Object

```
{
  "bsonType": "object",
  "required": ["personal_info", "financial_info", "
     academic_info", "loan_details", "references", "
     status", "application_date", "declaration", "
     signature", "username", "_id"],
  "properties": {
    "personal_info": {
      "bsonType": "object",
      "properties": {
        "full_name": {
          "bsonType": "string"
        },
        "dob": {
          "bsonType": "date"
        },
        "gender": {
          "bsonType": "string"
        },
        "nationality": {
          "bsonType": "string"
        },
        "marital_status": {
          "bsonType": "string"
        },
        "phone_number": {
          "bsonType": "string"
        },
        "email_address": {
          "bsonType": "string"
        },
        "residential_address": {
          "bsonType": "string"
        },
```

```
        "permanent_address": {
          "bsonType": "string"
        }
      }
    },
    "financial_info": {
      "bsonType": "object",
      "properties": {
        "total_family_income": {
          "bsonType": "double"
        },
        "other_income_sources": {
          "bsonType": "array",
          "items": {
            "bsonType": "string"
          }
        },
        "outstanding_loans_or_debts": {
          "bsonType": "array",
          "items": {
            "bsonType": "string"
          }
        }
      }
    },
    "academic_info": {
      "bsonType": "object",
      "properties": {
        "current_education_level": {
          "bsonType": "string"
        },
        "college_or_university": {
          "bsonType": "string"
        },
        "student_id": {
          "bsonType": "string"
        },
        "program_name_degree": {
          "bsonType": "string"
        },
```

```
        "duration_of_course": {
          "bsonType": "string"
        },
        "year_or_semester": {
          "bsonType": "string"
        },
        "gpa": {
          "bsonType": "double"
        },
        "achievements_or_awards": {
          "bsonType": "array",
          "items": {
            "bsonType": "string"
          }
        }
      }
    },
    "loan_details": {
      "bsonType": "object",
      "properties": {
        "loan_amount_requested": {
          "bsonType": "double"
        },
        "purpose_of_loan": {
          "bsonType": "string"
        },
        "proposed_repayment_period": {
          "bsonType": "string"
        },
        "preferred_repayment_frequency": {
          "bsonType": "string"
        }
      }
    },
    "references": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "properties": {
          "name": {
```

```
            "bsonType": "string"
          },
          "designation": {
            "bsonType": "string"
          },
          "contact_details": {
            "bsonType": "string"
          },
          "comments": {
            "bsonType": "string"
          }
        }
      }
    },
    "status": {
      "bsonType": "string"
    },
    "application_date": {
      "bsonType": "date"
    },
    "declaration": {
      "bsonType": "string"
    },
    "signature": {
      "bsonType": "string"
    },
    "username": {
      "bsonType": "string"
    },
    "_id": {
      "bsonType": "string"
    }
  }
}
```

Plan Object

```
{
  "bsonType": "object",
  "required": ["_id", "total_loan_amount", "start_date",
     "end_date", "repayment_frequency", "
```

```
      installment_amount", "reasoning", "application_id"],
  "properties": {
    "_id": {
      "bsonType": "string"
    },
    "total_loan_amount": {
      "bsonType": "double"
    },
    "start_date": {
      "bsonType": "date"
    },
    "end_date": {
      "bsonType": "date"
    },
    "repayment_frequency": {
      "bsonType": "string"
    },
    "installment_amount": {
      "bsonType": "double"
    },
    "reasoning": {
      "bsonType": "string"
    },
    "application_id": {
      "bsonType": "string"
    }
  }
}
```

RiskAssessment Object

```
{
  "bsonType": "object",
  "required": ["application_id", "financial_risk", "
     academic_risk", "personal_risk", "reference_risk", "
     repayment_potential", "created_at"],
  "properties": {
    "application_id": {
      "bsonType": "string"
    },
    "financial_risk": {
```

```
      "bsonType": "object",
      "properties": {
        "risk_score": {
          "bsonType": "double"
        },
        "calculations": {
          "bsonType": "string"
        }
      }
    },
    "academic_risk": {
      "bsonType": "object",
      "properties": {
        "risk_score": {
          "bsonType": "double"
        },
        "calculations": {
          "bsonType": "string"
        }
      }
    },
    "personal_risk": {
      "bsonType": "object",
      "properties": {
        "risk_score": {
          "bsonType": "double"
        },
        "calculations": {
          "bsonType": "string"
        }
      }
    },
    "reference_risk": {
      "bsonType": "object",
      "properties": {
        "risk_score": {
          "bsonType": "double"
        },
        "calculations": {
          "bsonType": "string"
```

```
          }
        }
      },
      "repayment_potential": {
        "bsonType": "object",
        "properties": {
          "risk_score": {
            "bsonType": "double"
          },
          "calculations": {
            "bsonType": "string"
          }
        }
      },
      "created_at": {
        "bsonType": "date"
      }
    }
}
```

User Object

```
{
  "bsonType": "object",
  "required": ["_id", "username", "email", "
     hashed_password", "role"],
  "properties": {
    "_id": {
      "bsonType": "string"
    },
    "username": {
      "bsonType": "string"
    },
    "email": {
      "bsonType": "string"
    },
    "hashed_password": {
      "bsonType": "string"
    },
    "role": {
      "bsonType": "string"
```

```
    },
    "documents": {
      "bsonType": "array",
      "items": {
        "bsonType": "object",
        "properties": {
          "page_content": {
            "bsonType": "string"
          },
          "metadata": {
            "bsonType": "object",
            "properties": {
              "source": {
                "bsonType": "string"
              },
              "author": {
                "bsonType": "string"
              },
              "section": {
                "bsonType": "string"
              },
              "document_type": {
                "bsonType": "string"
              },
              "date": {
                "bsonType": "date"
              }
            }
          }
        }
      }
    }
  }
}
```

Transaction Schema:

```
{
  "bsonType": "object",
  "required": ["transaction_id", "amount", "date", "
     donor_id", "loan_id", "student_id"],
```

```
  "properties": {
    "transaction_id": {
      "bsonType": "objectId"
    },
    "amount": {
      "bsonType": "decimal"
    },
    "date": {
      "bsonType": "date"
    },
    "donor_id": {
      "bsonType": "objectId"
    },
    "loan_id": {
      "bsonType": "objectId"
    },
    "student_id": {
      "bsonType": "objectId"
    }
  }
}
```

3.3 Domain Model

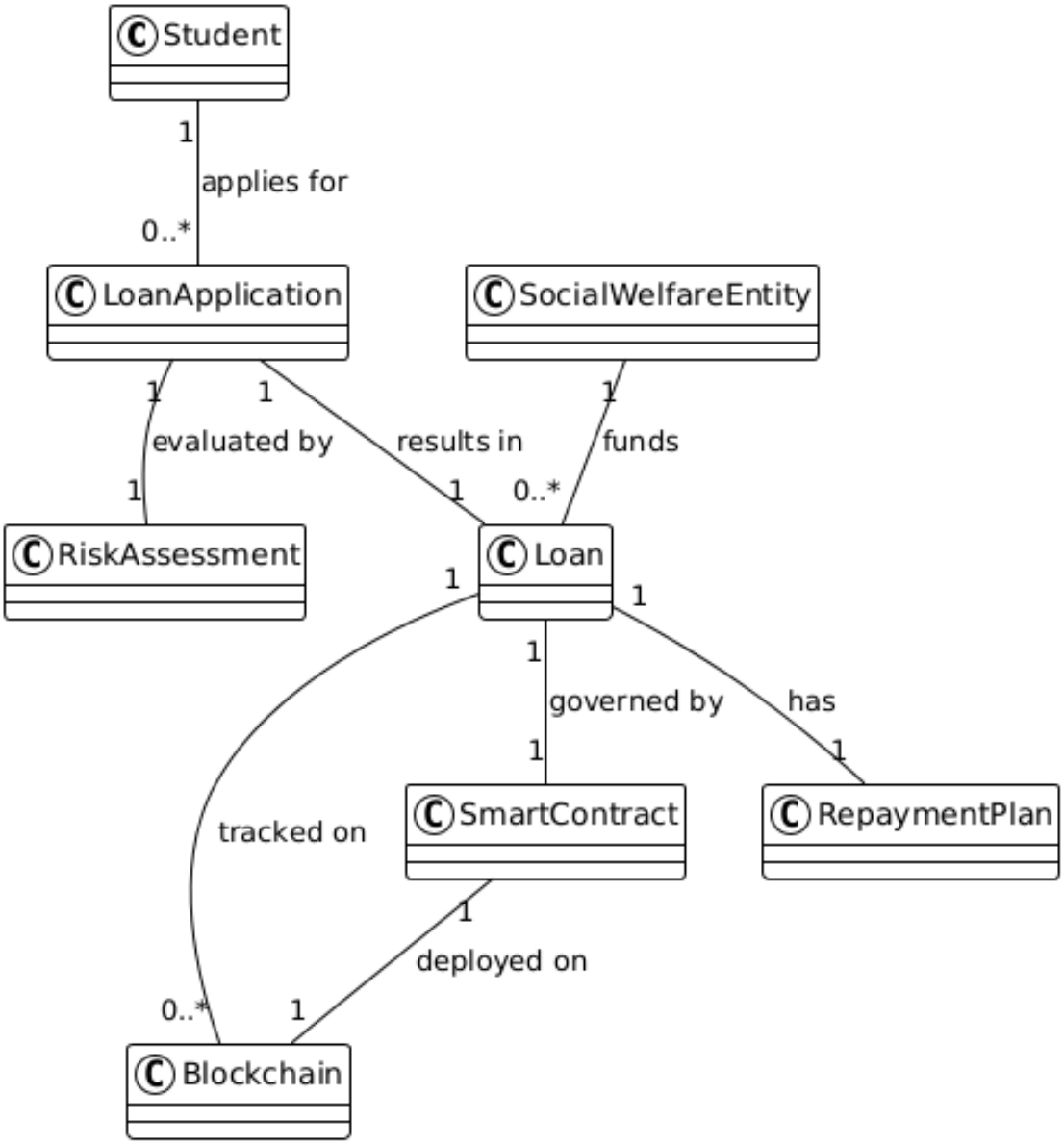


Figure 3.3: ScholarChain Domain Model

3.4 Design Models

3.4.1 Activity Diagram

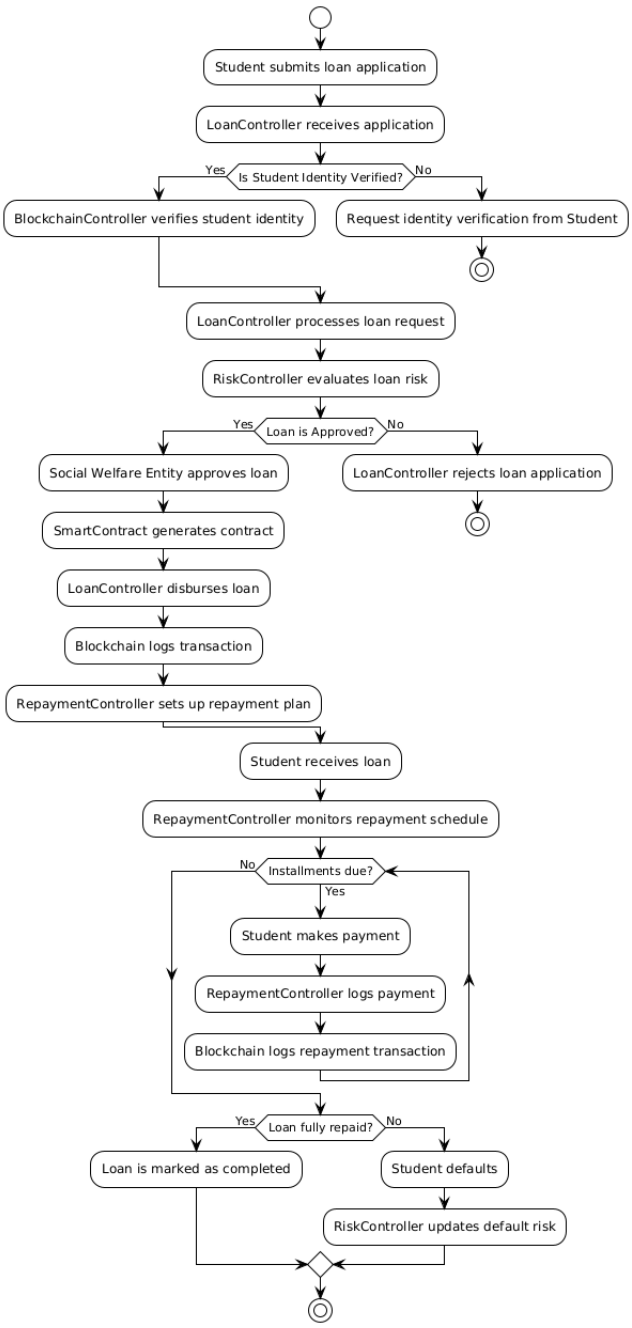


Figure 3.4: activity Diagram

The Activity Diagram details the workflow of a student's journey through the Scholar-Chain platform.

3.4.2 Class Diagram

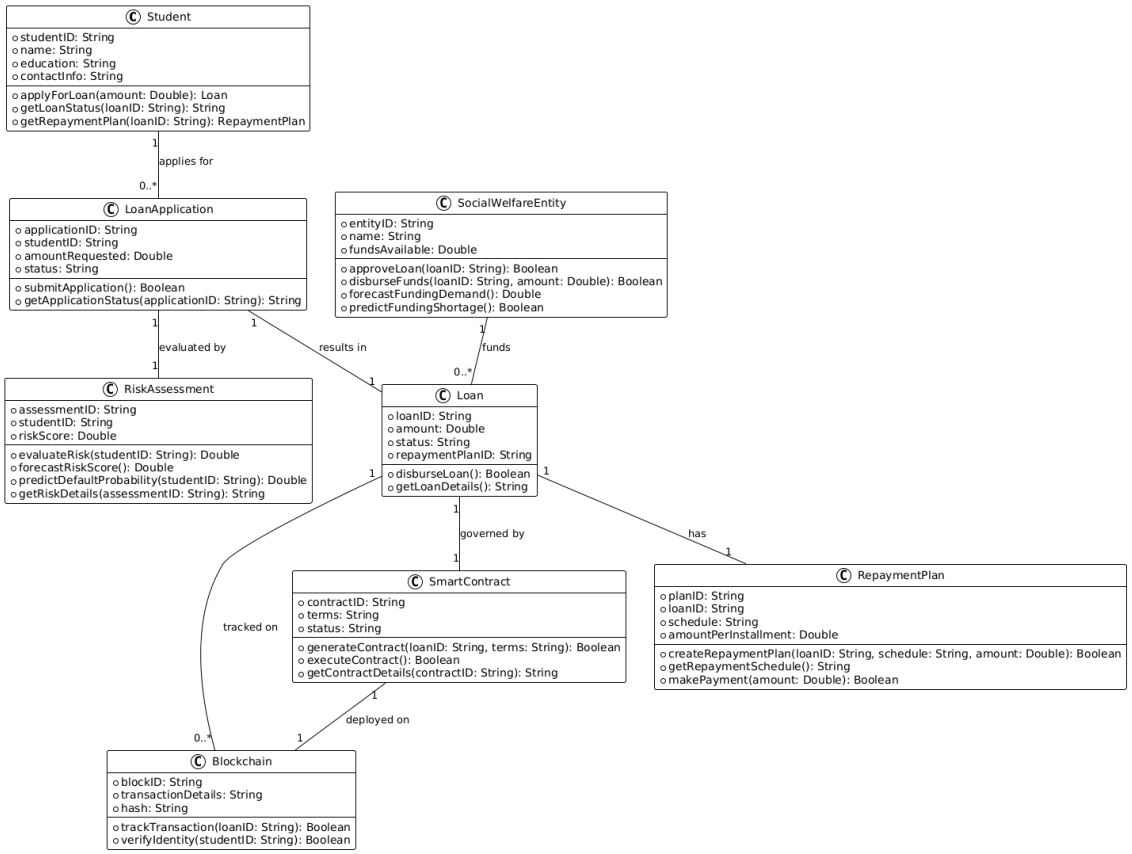


Figure 3.5: Class Diagram

The Class Diagram shows how the system's components are modeled as classes in an object-oriented design.

3.4.3 Sequence Diagrams

3.4.3.1 Loan Application Sequence Diagram

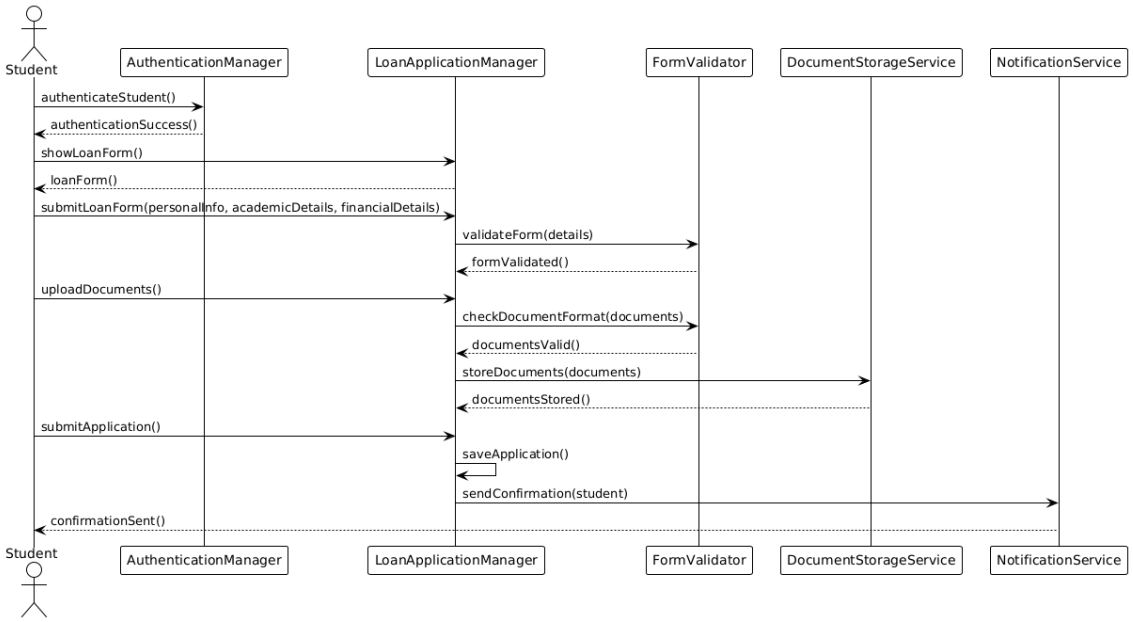


Figure 3.6: Loan Application Sequence Diagram

3.4.3.2 Funding Contribution Sequence Diagram

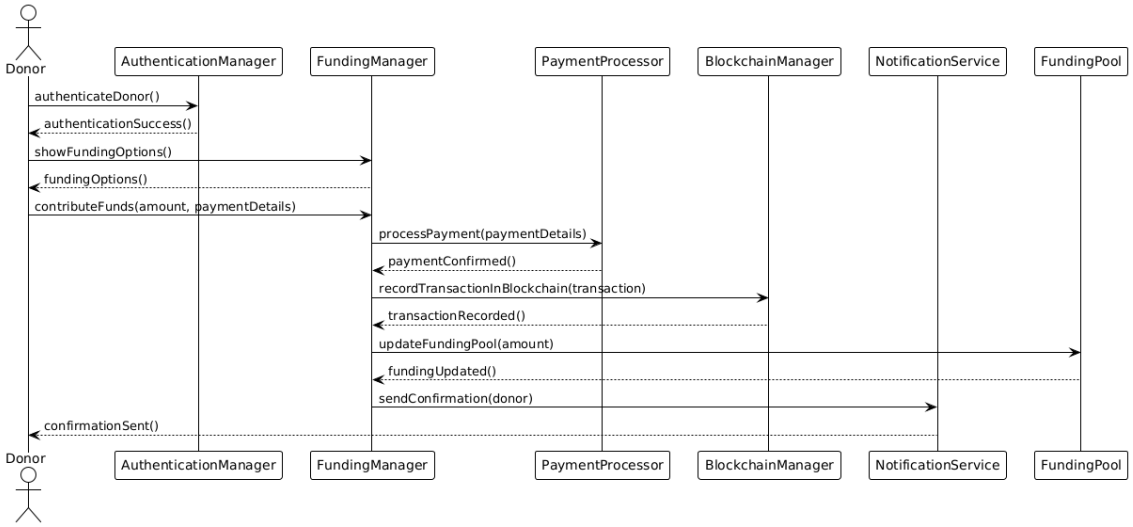


Figure 3.7: Funding Contribution Sequence Diagram

3.4.3.3 Approve Loan Sequence Diagram

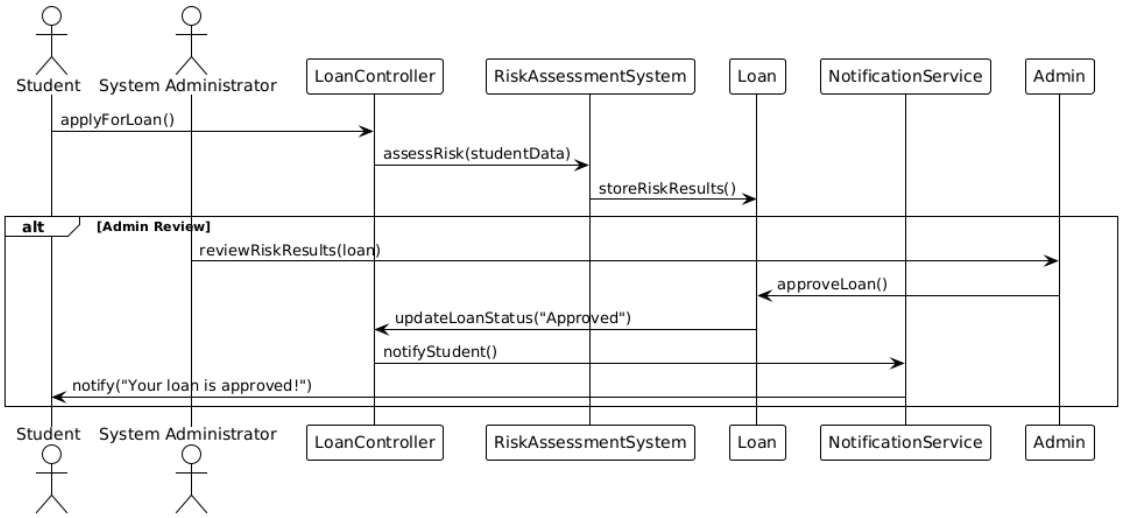


Figure 3.8: Approve Loan Sequence Diagram

3.4.3.4 Track Repayment Sequence Diagram

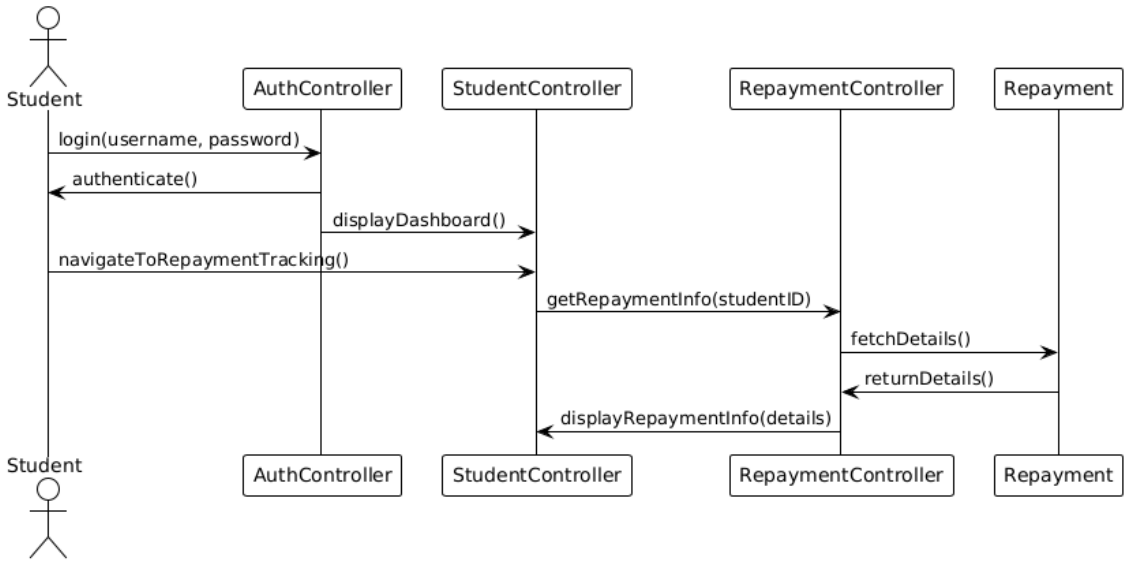


Figure 3.9: Track Repayment Sequence Diagram

3.4.3.5 Customer Support Sequence Diagram

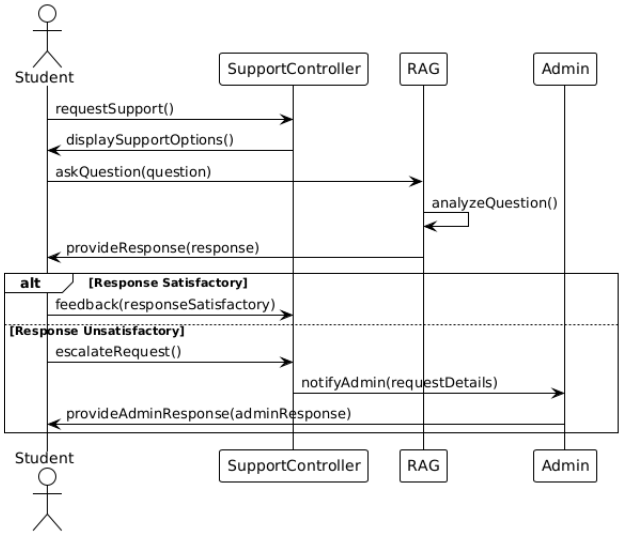


Figure 3.10: Customer Support Sequence Diagram

3.4.3.6 Payment Scheduling and Notification Sequence Diagram

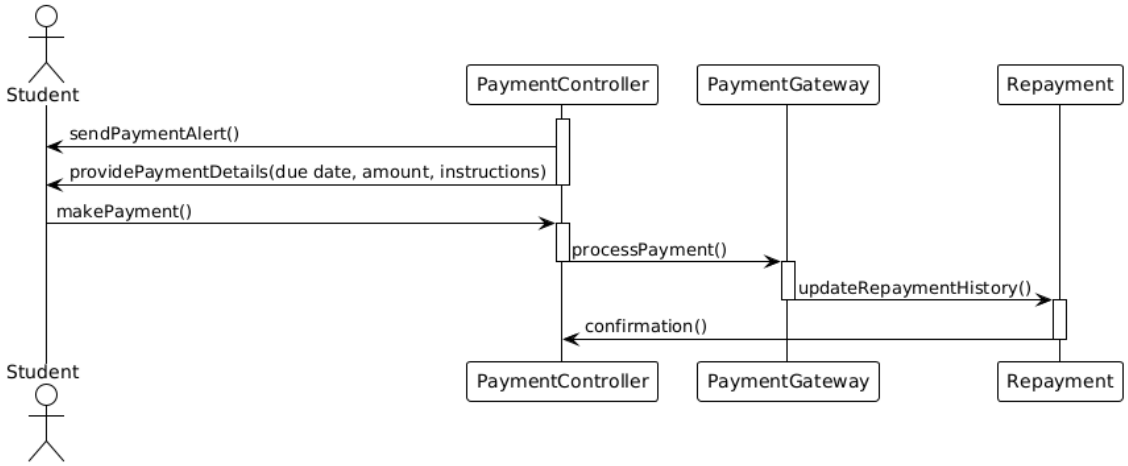


Figure 3.11: Payment Scheduling and Notification Sequence Diagram

3.4.3.7 Personalized Repayment Sequence Diagram

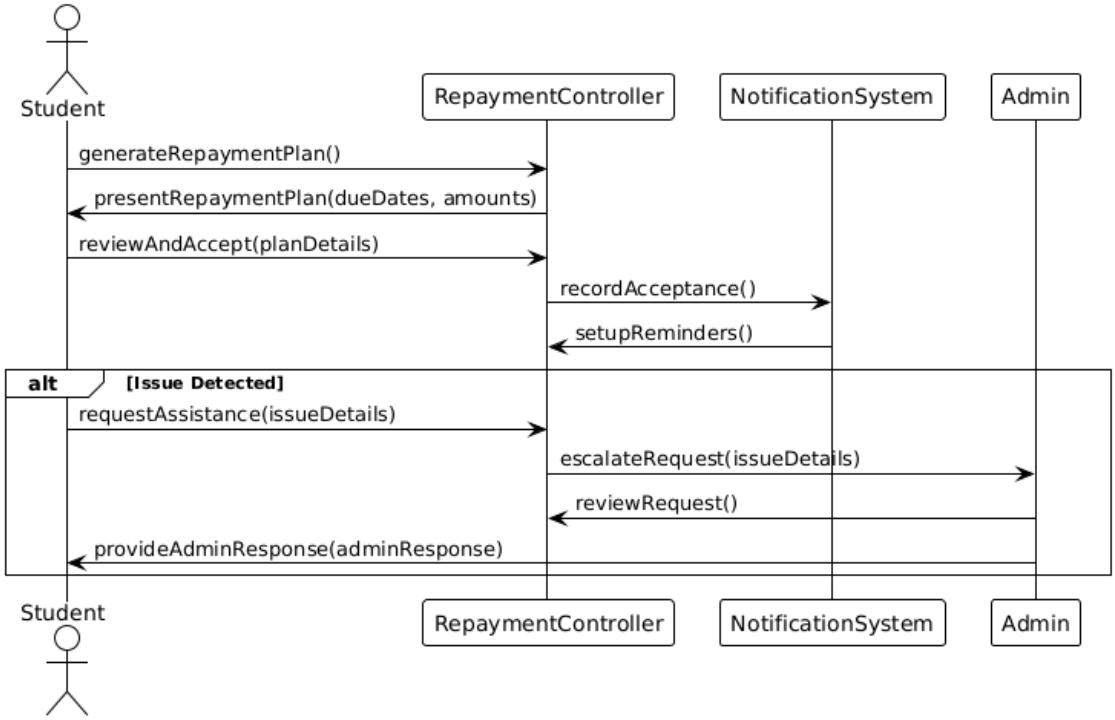


Figure 3.12: Personalized Repayment Sequence Diagram

3.4.3.8 Automated Loan Disbursement Sequence Diagram

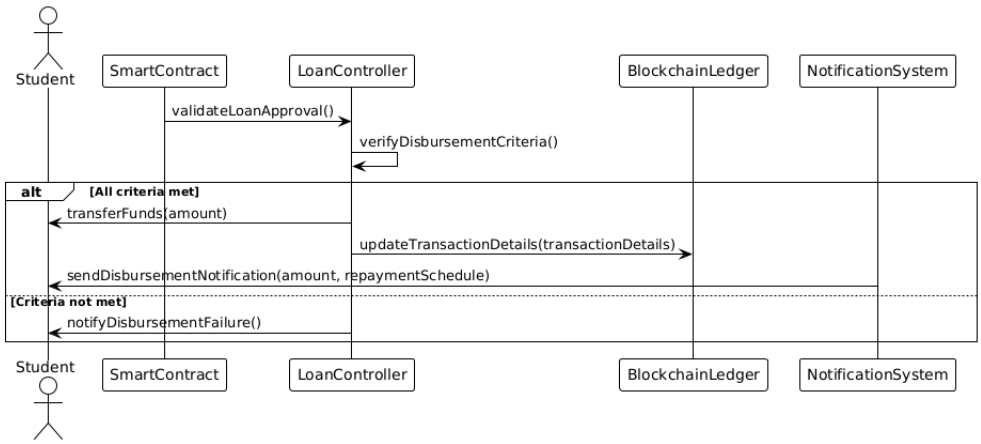


Figure 3.13: Automated Loan Disbursement Sequence Diagram

3.4.3.9 Verify Loan Sequence Diagram

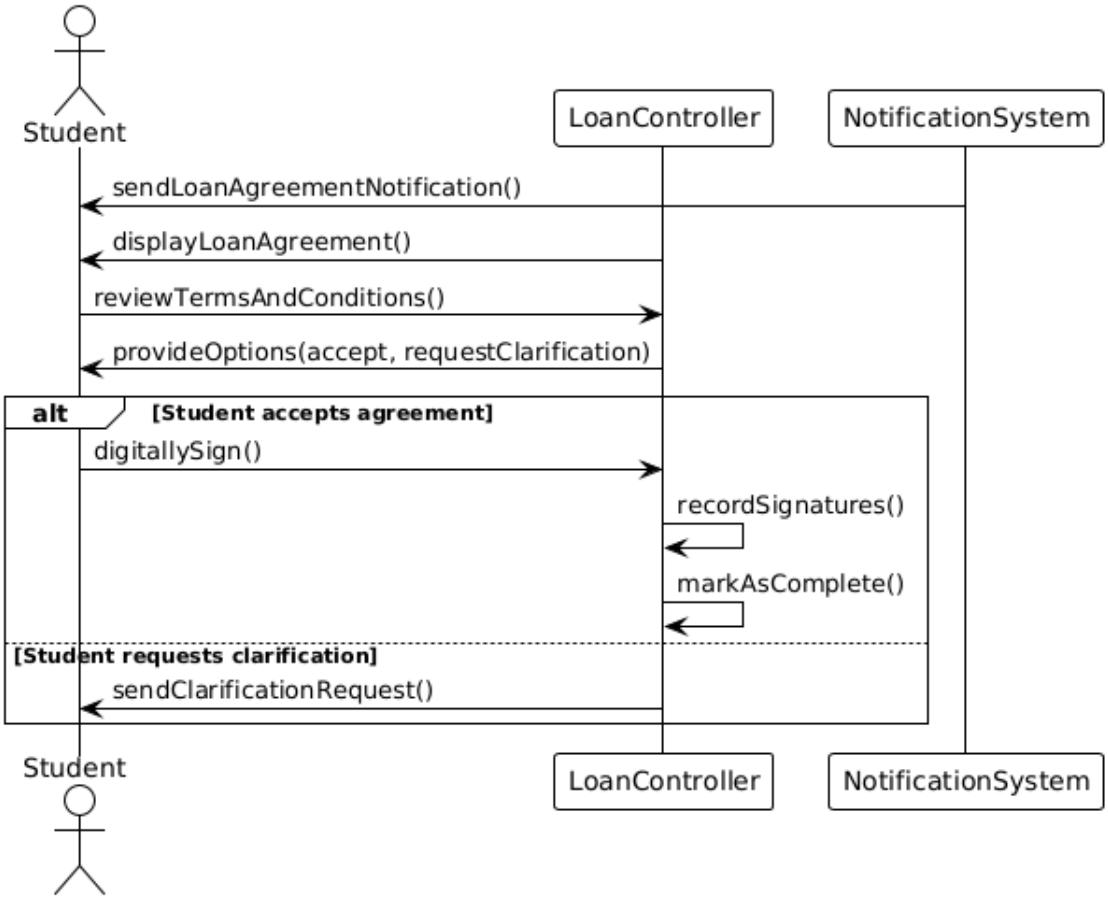


Figure 3.14: Verify Loan Sequence Diagram

3.4.4 State Transition Diagram

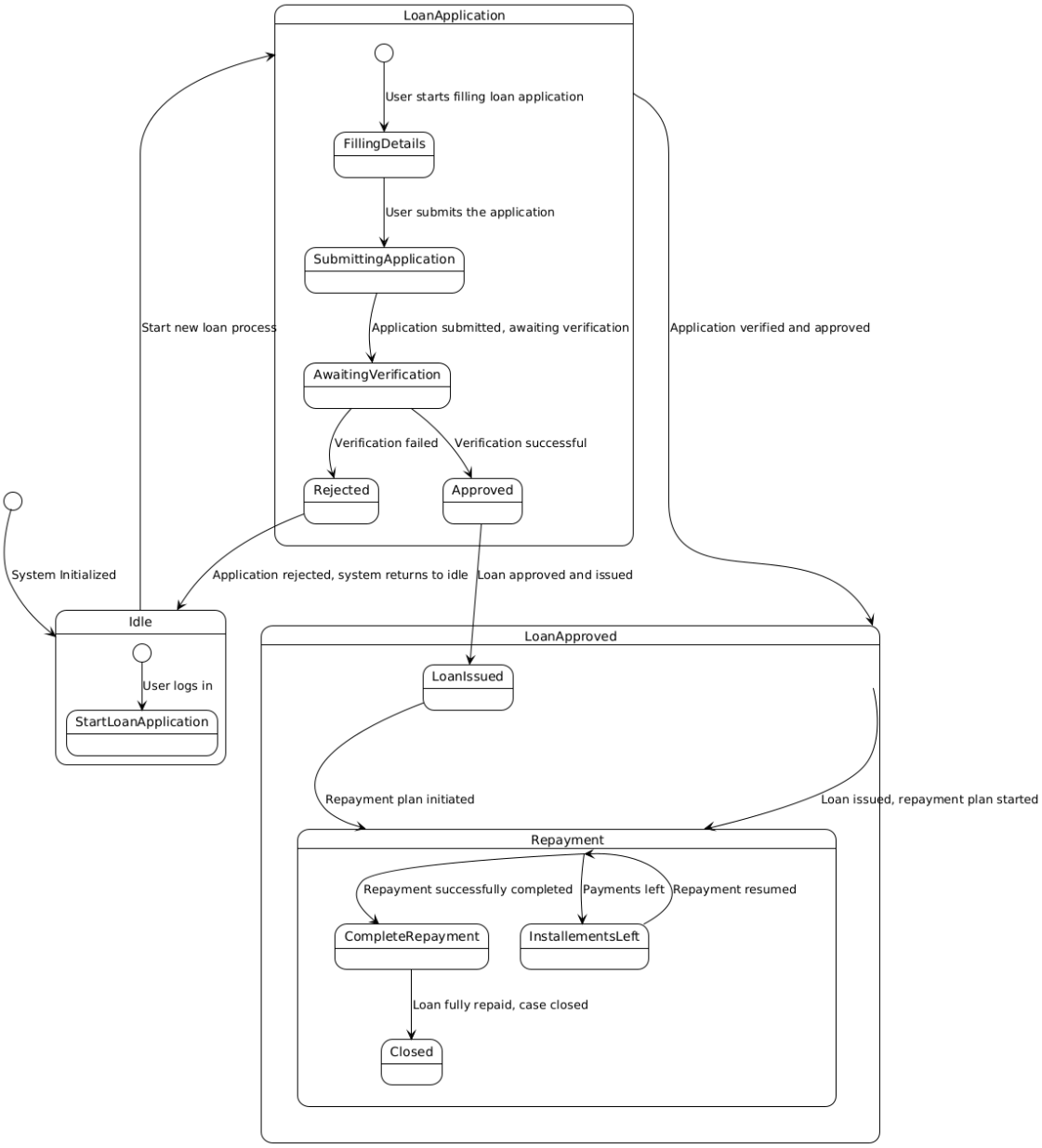


Figure 3.15: State Transition Diagram

The State Transition Diagram describes how an application progresses through different states.

Chapter 4

Implementation and Testing

4.1 Algorithm Design

Algorithm 1 User Registration

```
1: function REGISTERUSER(user_data)
2:     if user_exists(user_data.username, user_data.email) then
3:         Raise "Username or email already exists."
4:     end if
5:     hashed_password = hash_password(user_data.password)
6:     new_user = {
7:             "username": user_data.username,
8:             "email": user_data.email,
9:             "hashed_password": hashed_password,
10:            "documents": user_data.documents,
11:            "role": user_data.role
12:    }
13:    Insert new_user into user_collection
14:    Return "User successfully registered."
15: end function
```

Algorithm 2 Authenticate User

```
1: Function AuthenticateUser(username, password):
2: user = FindOne user_collection where username = username
3: if user is None or not verify_password(password, user.hashed_password)
   then
4:     Return None
5: end if
6: Return user
```

Algorithm 3 Verify Application

```
Function VerifyApplication(username):
update_data = { "status": "verified", "updated_at": datetime.utcnow() }
result = UpdateOne application_collection where username = username set update_data
if result.modified_count = 0 then
    Raise HTTPException(404, "Application not found or no fields to update.")
end if
Return "Application successfully verified."
```

Algorithm 4 Update Application

```
Function UpdateApplication(username, update_data):
if update_data is empty then
    Raise HTTPException(400, "No data provided for update.")
end if
update_data["updated_at"] = datetime.utcnow()
result = UpdateOne application_collection where username = username set update_data
if result.modified_count = 0 then
    Raise HTTPException(404, "Application not found or no fields to update.")
end if
Return "Application successfully updated."
```

Algorithm 5 Create Application

```
Function CreateApplication(application_data):
application_data["created_at"] = datetime.utcnow()
application_data["updated_at"] = datetime.utcnow()
result = InsertOne application_collection application_data
Return { "message": "Application successfully created.", "application_id": result.inserted_id }
```

Algorithm 6 Add Documents to User

```
Function AddDocuments(username, new_documents):
if new_documents = Empty then
    Return "No documents provided."
end if
result = Update user_collection WHERE username = username
Add new_documents to "documents" field in user document
if result.modified_count > 0 then
    Return "Documents successfully added."
else
    Return "Failed to add documents."
end if
```

Algorithm 7 Calculate Personal Information Risk Score

```
Function GetPersonalInformationScore(personal_info, risk_assessment):
score = 0
age = CalculateAge(personal_info.dob)
if 18 < age < 35 then
    score += 5
else
    score += 2.5
end if
Adjust score based on marital status:
if personal_info.marital_status = "Single" then
    score += 5 "Divorced"
    score += 3.5
else
    score += 2
end if
prompt = GeneratePrompt(personal_info)
response = RunMistral(prompt)
score += response["risk_score"]
Update risk_assessment.personal_risk with response
Return score
```

Algorithm 8 Calculate Academic Risk Score

```
Function GetAcademicRiskScore(academic_info, risk_assessment):
prompt = GeneratePrompt(academic_info)
response = RunMistral(prompt)
risk_assessment.academic_risk = ConvertToRiskScoreReasoning(response)
Return response["risk_score"]
```

Algorithm 9 Calculate Financial Risk Score

```
Function GetFinancialRiskScore(financial_info, loan_details, risk_assessment):
TFI = CalculateMonthlyIncome(financial_info.total_family_income)
if TFI < 100,000 then
    score = 2
else if TFI > 250,000 then
    score = 8
else
    score = 15
end if
prompt = GeneratePrompt(financial_info, loan_details)
response = RunMistral(prompt)
score += response["risk_score"]
Update risk_assessment.financial_risk with response
Return score
```

Algorithm 10 Retrieve and Generate (RAG)

Input: Query q, Temporary Directory $temp_dir$
Output: Generated response
Load `vectorstore` from $temp_dir$ using embedding model
Initialize retriever with `vectorstore`
$retrieved_docs \leftarrow$ retriever.retrieve(q)
Format $retrieved_docs$ as context
Load `LLM (Language Model)`
Define prompt for retrieval-augmented generation
Initialize RAG chain with retriever, prompt, and LLM
$response \leftarrow$ RAG chain.generate(q)
Return $response$

Algorithm 11 Create Vector Store

Input: Username u, New Document Flag $newDoc$
Output: Path to temporary vector store directory
if u exists in vector_stores and $newDoc$ is True **then**
    Delete existing vector store for u
    Remove u from vector_stores
end if
$user \leftarrow$ Get user documents by username u
if no documents exist for u **then**
    `Raise` "No documents found for user."
end if
Convert $user.documents$ to Document objects
Split documents into smaller chunks using text splitter
Initialize embedding model
Create vector store from document chunks
Persist vector store in a temporary directory
Store directory path in vector_stores for u
Return Path to vector store directory

Algorithm 12 Generate Personalised Loan Repayment Plan

Input: Current user data *current_user*
Output: Personalised Loan Repayment Plan (JSON Object)
Retrieve application data *application_dict* for *current_user.username*
Check if a repayment plan exists for *application_dict*["*id*"]
if existing plan exists **then**
    Convert *existing_plan*["*id*"] to string
    Return *existing_plan*
end if
Create `Application` object from *application_dict*
Retrieve a temporary vector store directory *temp_dir* for *current_user.username*
Define *query* to provide loan repayment plan based on:
        Loan amount, academic details, financial transactions, and preferences
        Include calculation constraints and JSON formatting rules
Send *query* to RAG bot using `LangChainService` and *temp_dir*
Parse RAG bot response *response*["*response*"] for repayment plan
Post-process *response* to ensure correctness
Parse processed response to JSON object *json_object*
Add *application_id* from *application_dict*["*id*"] to *json_object*
Save *json_object* to database as a new plan
Convert *json_object*["*id*"] to string
Return *json_object* JSON decoding error
Log error and raise HTTP 500 with error details

4.2 External APIs/SDKs

Describe the third-party APIs/SDKs used in the project implementation in the following table. Few examples of APIs are provided in the table.

API/SDK and Version	Description	Purpose of Usage	API Endpoint/Function/-Class Used
FastAPI (v0.95.2)	Web framework for building APIs	Create RESTful endpoints for interacting with Mistral and MongoDB	`@app.get()`, `@app.post()`
Mistral API (Pixtral)	Large language model API	Generate responses and predictions based on user input	`POST /predict`, `GET /status`
LangChain (v0.0.221)	Framework for building LLM-powered applications	Chain LLM calls with prompts and memory for conversational AI	`LLMChain`, `OpenAI`, `PromptTemplate`
PyMongo (v4.5.0)	MongoDB driver for Python	Perform synchronous CRUD operations on MongoDB	`MongoClient()`, `collection.insert_one()`, `collection.find_one()`
Motor (v3.1.0)	Asynchronous MongoDB driver for Python	Perform asynchronous CRUD operations on MongoDB	`AsyncIOMotorClient()`, `await collection.find_one()`, `await collection.insert_one()`
Requests (v2.31.0)	HTTP library for Python	Send HTTP requests to Mistral API	`requests.post()`, `requests.get()`

Table 4.1: List of APIs/SDKs used in the project

4.3 Testing Details

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended.

4.3.1 Unit Testing

Each unit test is designed to test a specific function or method independently from other components, helping to identify issues directly related to the functionality being tested.

gray!30 **Test Case ID**	**Test Objective**	**Precondition**	**Steps**	**Test Data**	**Expected Result**	**Post-Condition**	**Actual Result**	**Pass/Fail**

TC001	Validate user creation with valid data in 'User' model	Ensure database is available and empty	1. Create a user with valid 'username', 'email', and 'hashed_password'.
			2. Save the user to the database.
			3. Retrieve user to verify.
				`_id: 1`
				`username: "testuser"`
				`email: "test@example.com"`
				`hashed_password: "hashedpass123"`
					The user should be created successfully and retrievable with matching data.	The user is stored in the database and is accessible via queries.	As expected, user is created and retrieved successfully.	Pass

TC002	Validate user creation fails with missing required fields	Database should be available.	1. Attempt to create a user without 'username' or 'email'.
			2. Catch validation error.
				`email: ""`
				`username: "testuser"`
				`hashed_password: "hashedpass123"`
					An error should be raised indicating missing required fields.	User should not be created in the database.	ValidationError was raised as expected	Pass

TC003	Validate login with correct credentials	User must already be registered with valid 'username' and 'hashed_password' in the database.	1. Submit login request with valid 'username' and 'password'. 2. Verify the hashed password. 3. Return a success message and token if valid.	`username: "testuser"` `password: "plaintextpass"`	The login should succeed, and an authentication token should be returned.	User is logged in, and token is generated.	As expected, login was successful and token was generated.	Pass
TC004	Validate login with incorrect credentials	User must already be registered with valid 'username' and 'hashed_password' in the database.	1. Submit login request with valid 'username' but incorrect 'password'. 2. Verify the hashed password fails. 3. Return an authentication error.	`username: "testuser"` `password: "wrongpass"`	The login should fail with an error indicating incorrect credentials.	User remains logged out, and no token is generated.	As expected, login failed with an incorrect password error.	Pass
TC005	Validate document upload with a valid PDF file	User must be logged in.	1. Select a valid PDF file for upload. 2. Submit the upload request. 3. Verify the file type. 4. Save the file and its metadata.	File: 'document.pdf' MIME type: 'application/pdf'	The PDF file should be uploaded and saved successfully.	The document is stored in the database and linked to the user.	As expected, PDF file was uploaded and saved successfully.	Pass
TC006	Validate document upload failure for non-PDF files	User must be logged in.	1. Select a non-PDF file for upload (e.g., '.docx' or '.jpg'). 2. Submit the upload request. 3. Verify the file type. 4. Reject the upload with an error message.	File: 'document.docx' MIME type: 'application/vnd.openxmlformats-officedocument.wordprocessingml.document'	The upload should be rejected with an error indicating only PDF files are allowed.	The document is not stored in the database, and an error message is returned.	As expected, non-PDF file upload was rejected with an appropriate error message.	Pass
TC007	Validate auto-filling of personal and academic details from uploaded PDF using Mistral LLM	PDF document containing personal and academic information must be uploaded.	1. Upload the PDF document. 2. Invoke RAG pipeline to extract relevant fields. 3. Auto-fill the 'PersonalInfo', 'AcademicInfo', 'FinancialInfo', 'Referals' sections of the application. 4. Verify the auto-filled fields against the PDF.	PDF containing: Full Name: "John Doe" DOB: "1995-05-10" Program Name: "B.Sc. Computer Science"	Personal and academic details should be auto-filled correctly in the application form.	Auto-filled details are saved in the application draft.	As expected, details were extracted and correctly auto-filled.	Pass
TC008	Validate failure case when required data is missing in uploaded document	PDF document uploaded without personal information fields.	1. Upload the PDF document. 2. Invoke RAG pipeline for field extraction. 3. Auto-filling fails due to missing critical fields like 'full$_n$*ame*'*or*'*dob*'. 4.*Returnanerrormessage.*	PDF missing 'full$_n$*ame*', '*dob*'.	empty strings returned and missing required fields should be displayed, and no auto-filling should occur.	Application remains empty or incomplete.	As expected, auto-filling failed, and an appropriate error message was displayed.	Pass
TC009	Validate risk score calculation for a complete application with all necessary documents using Mistral LLM	The application must be fully submitted with all required documents (personal, financial, academic, and references).	1. Submit the complete application. 2. Pass the application and uploaded documents as context to Mistral LLM. 3. Invoke the LLM to calculate the risk score for all categories. 4. Verify the calculated scores and reasoning.	Application with: Financial info Academic info References Personal info	Risk scores for all categories (financial, academic, personal, reference, repayment potential) should be calculated with proper reasoning.	Risk scores and reasoning are saved in the database.	As expected, risk scores calculated and saved successfully.	Pass
TC010	Validate risk score calculation for an application with a weaker case where the risk score is below the approval threshold	The application is submitted with complete information, but the financial, academic, or repayment aspects show a weaker case.	1. Submit the complete application. 2. Pass the application and documents to Mistral LLM. 3. Invoke the LLM to calculate the risk score. 4. Verify that the calculated score is below the approval threshold, and a rejection recommendation is provided.	Application with complete fields but weaker financial stability or low GPA (e.g., total family income below minimum, GPA less than 2.5).	The risk score should be calculated below the threshold (e.g., less than 75), and a rejection recommendation should be given with detailed reasoning.	The system stores the rejection recommendation and risk score reasoning in the database.	As expected, the system calculated a lower risk score and provided a rejection recommendation.	Pass

Table 4.2: Unit Test Cases for Loan Applications Part 1

gray!30 **Test Case ID**	**Test Objective**	**Precondition**	**Steps**	**Test Data**	**Expected Result**	**Post-Condition**	**Actual Result**	**Pass/Fail**
TC011	Validate the repayment plan generation considering available funds and approved loan amount in PKR	The application has been approved with a high risk score, and available system funds are greater than or equal to the requested loan amount.	1. Submit the approved application. 2. Check available system funds. 3. Pass the application and funds data to the repayment plan generator. 4. Generate a repayment plan with details like start date, end date, repayment frequency, and installment amount. 5. Verify that the plan matches the loan details and available funds.	Approved application with: Total loan amount: PKR 2,000,000 Repayment frequency: Monthly Available system funds: PKR 5,000,000	A repayment plan is generated, considering the available funds, with correct dates, frequency, installment amount, and reasoning for the approved loan in PKR.	The repayment plan is saved in the database and linked to the application ID, and funds are reserved for the loan.	As expected, a valid repayment plan was generated considering the available funds and stored successfully.	Pass
TC012	Validate repayment plan generation with custom repayment frequency and consideration of available funds in PKR	The application has been approved with a higher risk score, and the applicant requests a custom repayment frequency and installment amount, with system funds being checked for availability.	1. Submit the approved application with custom repayment preferences. 2. Check available system funds. 3. Pass the application and custom preferences to the repayment plan generator. 4. Generate the repayment plan. 5. Verify that the plan reflects custom preferences while ensuring compliance with loan terms and available funds.	Approved application with: Total loan amount: PKR 3,000,000 Custom frequency: Quarterly Installment amount: PKR 750,000 Available system funds: PKR 4,000,000	A repayment plan is generated considering custom frequency, installment preferences, and available funds, with proper reasoning documented in PKR.	The custom repayment plan is saved and associated with the application ID in the database, and funds are allocated for the loan.	As expected, the repayment plan with custom preferences and available funds was generated and stored successfully.	Pass
TC013	Verify admin user login and access to application details, risk score, and repayment plan	Admin user is registered and has valid credentials. Application, risk score, and repayment plan are generated and stored in the system.	1. Admin logs in using valid credentials. 2. Navigate to the application dashboard. 3. Select a submitted application. 4. Review application fields, calculated risk score, and generated repayment plan. 5. Verify the correctness of all details displayed.	Admin credentials: Username: admin123 Password: Admin@123 Application ID: APP-1001	Admin successfully logs in and accesses all application details, including risk score and repayment plan, ensuring their accuracy.	Admin is able to view, verify, and manage the application, risk score, and repayment plan without errors.	As expected, all details were accessible and accurate upon review by the admin.	Pass
TC014	Verify admin user ability to approve or reject an application based on risk score and repayment plan	Admin user is logged in. The application has a risk score and a generated repayment plan.	1. Admin logs in. 2. Navigate to the application dashboard. 3. Select an application with a generated risk score and repayment plan. 4. Review the risk score and repayment plan. 5. Approve or reject the application based on the provided data. 6. Confirm the action.	Admin credentials: Username: admin123 Password: Admin@123 Application ID: APP-1002 Risk Score: 85 Loan Amount: PKR 1,500,000	Admin is able to review the application, risk score, and repayment plan, and successfully approve or reject the application with reasoning recorded.	The system records the admin's decision and updates the application's status, along with reasoning for approval or rejection.	As expected, admin reviewed and approved/rejected the application successfully, and the status was updated.	Pass

Table 4.3: Unit Test Cases for Loan Applications Part 2

Chapter 5

Conclusions and Future Work

5.1 Conclusion

In this report, we have introduced **ScholarChain**, a comprehensive platform that seamlessly integrates artificial intelligence and blockchain technologies to address the challenges of interest-free student lending. Beginning with a clear problem statement—providing transparent, accessible, and personalized financial support to students—we designed and implemented a modular system architecture comprising:

- **AI Module:** A multi-stage risk assessment engine that combines financial data and psychometric inputs, coupled with a Retrieval-Augmented Generation (RAG) chatbot for contextual borrower support and repayment plan personalization.
- **Blockchain Module:** An Ethereum-based implementation of immutable smart contracts for loan agreements, disbursements, and repayments, ensuring auditability and donor confidence.
- **Web Application:** Role-based dashboards for students and donors, real-time notifications, and administrative audit trails, built on FastAPI and NextJS.

Through detailed algorithm design, JSON-schemadriven data modeling, and extensive unit tests, we demonstrated that ScholarChain can accurately assess borrower risk, automate contract execution, and generate tailored repayment schedules. Prototype evaluations indicate that our hybrid scoring approach effectively differentiates borrower profiles, while the blockchain backbone preserves the transparency and integrity of transactions. Moreover, informal usability feedback highlights the platform's intuitive interfaces and its potential to foster trust among both students and donors.

Nonetheless, the current implementation is a proof-of-concept. Limitations include the need for larger, more diverse datasets to validate psychometric scoring, performance

benchmarking under production-scale workloads, and regulatory compliance at jurisdictional levels. Despite these constraints, ScholarChain lays a solid foundation for next-generation, socially impactful lending platforms that harness the synergy of AI and DLT.

5.2 Future Work

1. **Refinement of Prompt Engineering for the AI Module**
    - *Dynamic Prompt Libraries:* Develop a repository of adaptive prompt templates tailored to various borrower profiles (e.g., STEM vs. humanities students, differing socioeconomic backgrounds).
    - *Iterative A/B Testing:* Integrate A/B testing frameworks that compare multiple prompt variants in live simulations, using quantitative metrics such as answer relevancy, response coherence, and borrower satisfaction.
2. **Tokenization and Ethereum Exchange Listing**
    - *ScholarChain Token Design:* Define tokenomics that balance incentives for donors (e.g., governance rights, staking rewards) and borrowers (e.g., loyalty discounts on repayment). Establish total supply, vesting schedules, and deflationary mechanisms if appropriate.
    - *Regulatory Roadmap:* Outline compliance steps for token issuance—KYC/AML protocols, legal opinions on securities classification, and necessary disclosures.
    - *Business Model & Partnerships:* Explore revenue streams such as transaction fees, token-based governance subscriptions, and yield-earning strategies by staking pooled tokens in DeFi protocols. Investigate partnerships with decentralized exchanges (DEXs) and centralized platforms to secure initial liquidity and market-making support.
3. **RealWorld Pilots with Educational Partners**
    - *Pilot Program Design:* Partner with universities, NGOs, or microfinance institutions to recruit a cohort of students and donors. Define clear success metrics: repayment rates, time-to-approval, user satisfaction scores, and social-impact indicators (e.g., increased enrollment in underserved regions).
    - *Iterative Feature Refinement:* Use pilot feedback to calibrate risk-scoring thresholds, adjust smart-contract terms, and optimize UI flows.
    - *Impact Assessment:* Conduct periodic surveys and data analyses to measure outcomes against control groups, generating empirical evidence of ScholarChain's efficacy and guiding future scaling decisions.

Bibliography

Appendix A

Appendices

A.1 Appendix A

A.1.1 Use Case Diagram example (Online Shopping System)

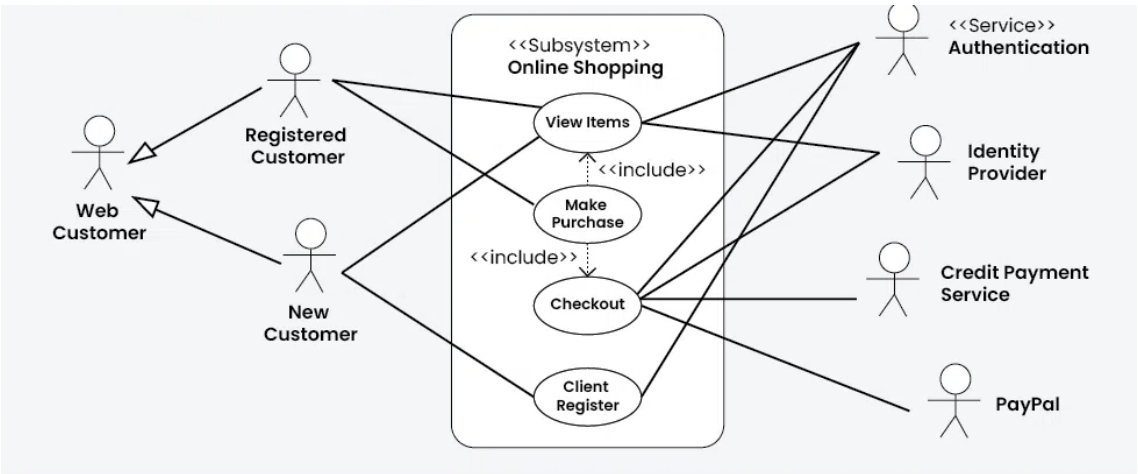


Figure A.1: Use Case Diagram for the Online Shopping System

A.1.2 Detail Use Case Example

ID	#1
Name	Overview elements
Short Description	All elements are shown in a list, where the user can set different filters.
Goal	Displaying elements in a changeable view.
Preconditions	The user got access to the system and is logged in. The user got the right to see schedules.
Success End Condition	The correct elements (filter and sorting) are displayed.
Fall End Condition	Elements not matching the criteria are displayed.
Stakeholder	Customer manager
Trigger	Login in as customer manager (because overview is on the starting screen for this role).
Normal Flow	1. The list of all elements matching default criteria are shown. 2. The user changes the filter criteria. 3. The user presses the Filter button. 4. The list shows all matching elements. Optional: 5. The user presses the Clear button. 6. The list of all elements matching default criteria are shown.
Alternative Flows	With click on the column headers, the list sorting can be changed. Different sorting for the different columns is described in the table below.
Includes	Login
Frequency of Use	About 50 times per day
Constraints and Special Requirements	None
Assumptions	None
Notes and Issues	None

Figure A.2: Detail Use Case Example

A.1.3 Event-Response Table for a Highway Intersection

Event	System State	Response
Road sensor detects vehicle entering left-turn lane.	Left-turn signal is red. Cross-traffic signal is green.	Start green-to-amber countdown timer for cross-traffic signal.
Green-to-amber countdown timer reaches zero.	Cross-traffic signal is green.	1. Turn cross-traffic signal amber. 2. Start amber-to-red countdown timer.
Amber-to-red countdown timer reaches zero.	Cross-traffic signal is amber.	1. Turn cross-traffic signal red. 2. Wait 1 second. 3. Turn left-turn signal green. 4. Start left-turn-signal countdown timer.
Pedestrian presses a specific walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is not activated.	Start walk-request countdown timer.
Pedestrian presses walk-request button.	Pedestrian sign is solid Don't Walk. Walk-request countdown timer is activated.	Do nothing.
Walk-request countdown timer reaches zero plus the amber display time.	Pedestrian sign is solid Don't Walk.	Change all green traffic signals to amber.
Walk-request countdown timer reaches zero.	Pedestrian sign is solid Don't Walk.	1. Change all amber traffic signals to red. 2. Wait 1 second. 3. Set pedestrian sign to Walk. 4. Start don't-walk countdown timer.

Figure A.3: Example of Event Response Table

A.1.4 Story Board Example For Android App

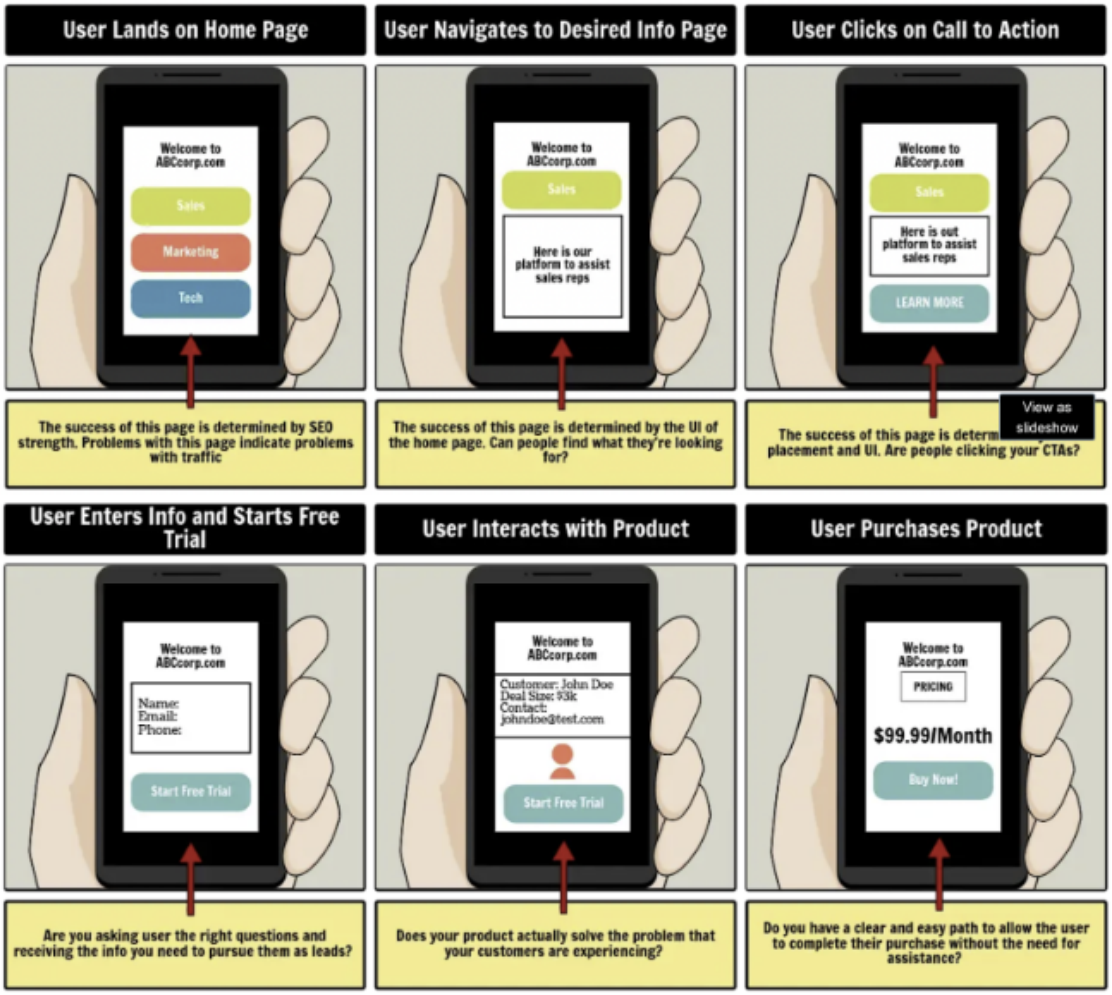


Figure A.4: Example of Story Board

A.2 Appendix B

A.2.1 Domain Model Example For Online Shopping

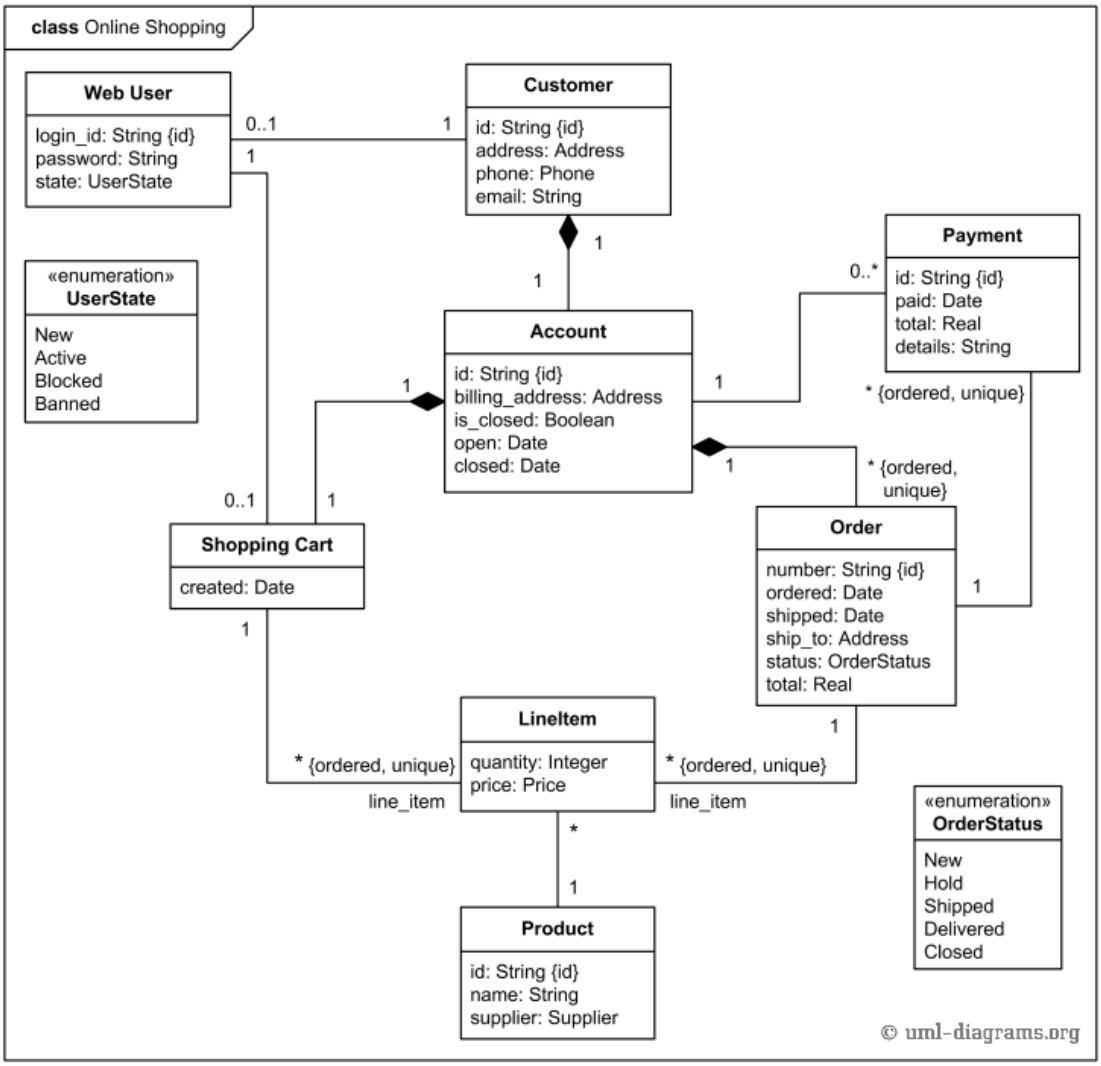


Figure A.5: Domain Model Example For Online Shopping Application

A.3 Appendix C

A.3.1 Box And Line Example For Online Shopping

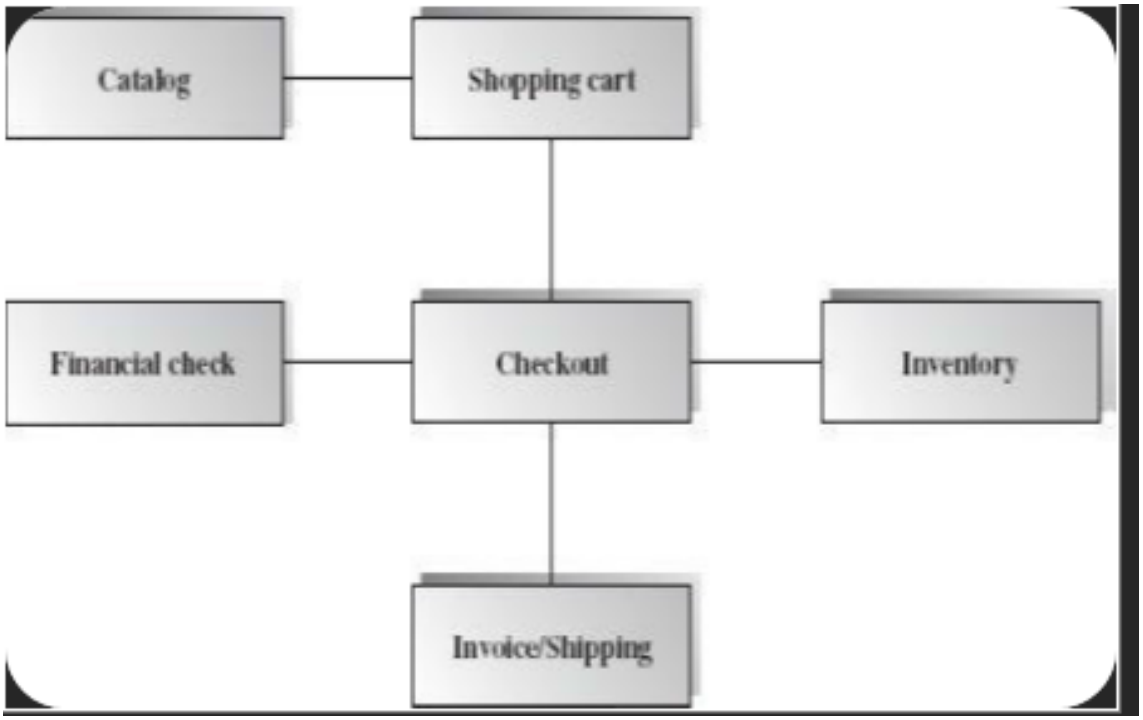


Figure A.6: Box and Line Diagram For Online Shopping Application

A.3.2 Architecture Pattern Example For Online Shopping

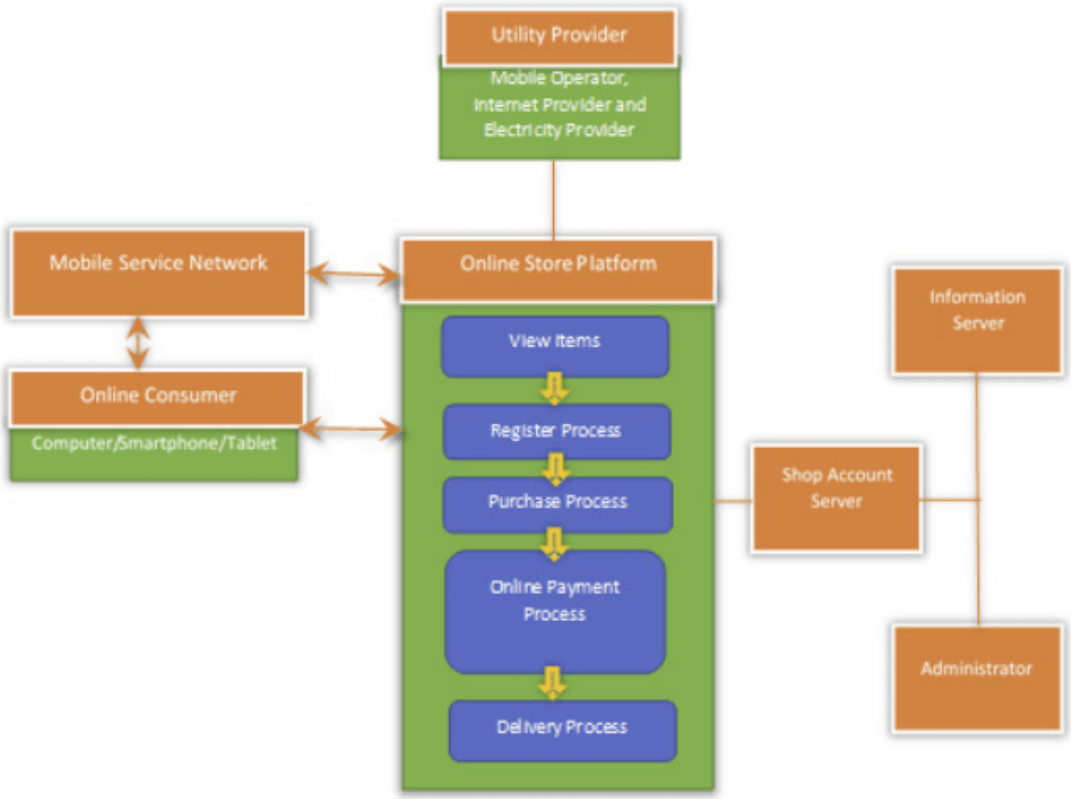


Figure A.7: Architecture Pattern For Online Shopping Application

A.4 Appendix D

A.4.1 Activity Diagram

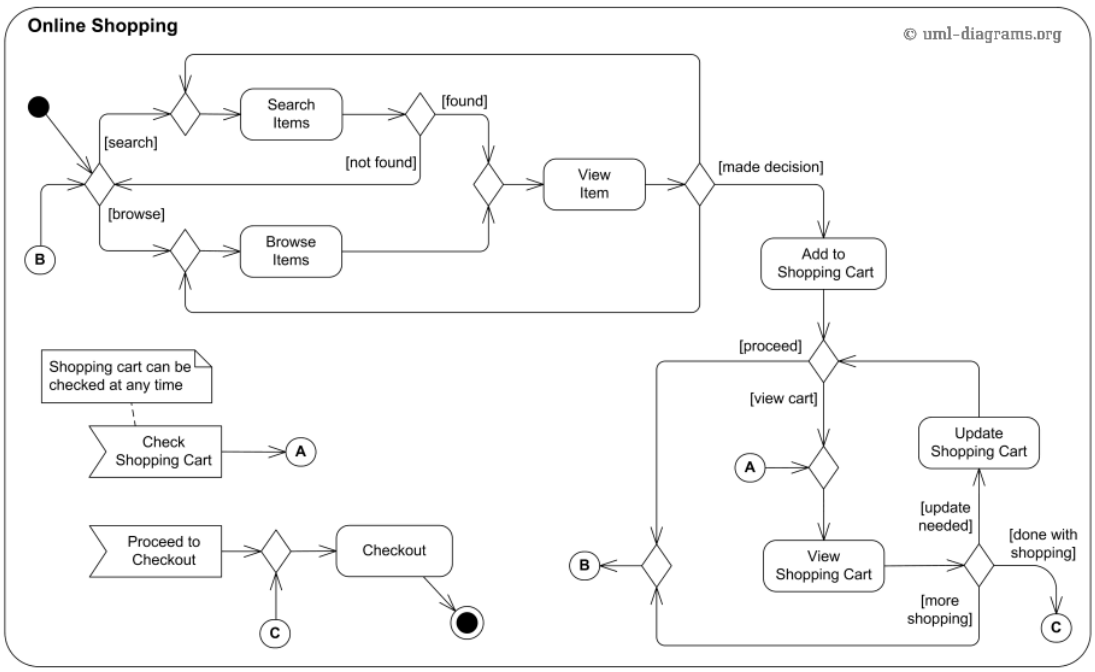


Figure A.8: Activity Diagram For Online Shopping Application

A.4.2 Class Diagram

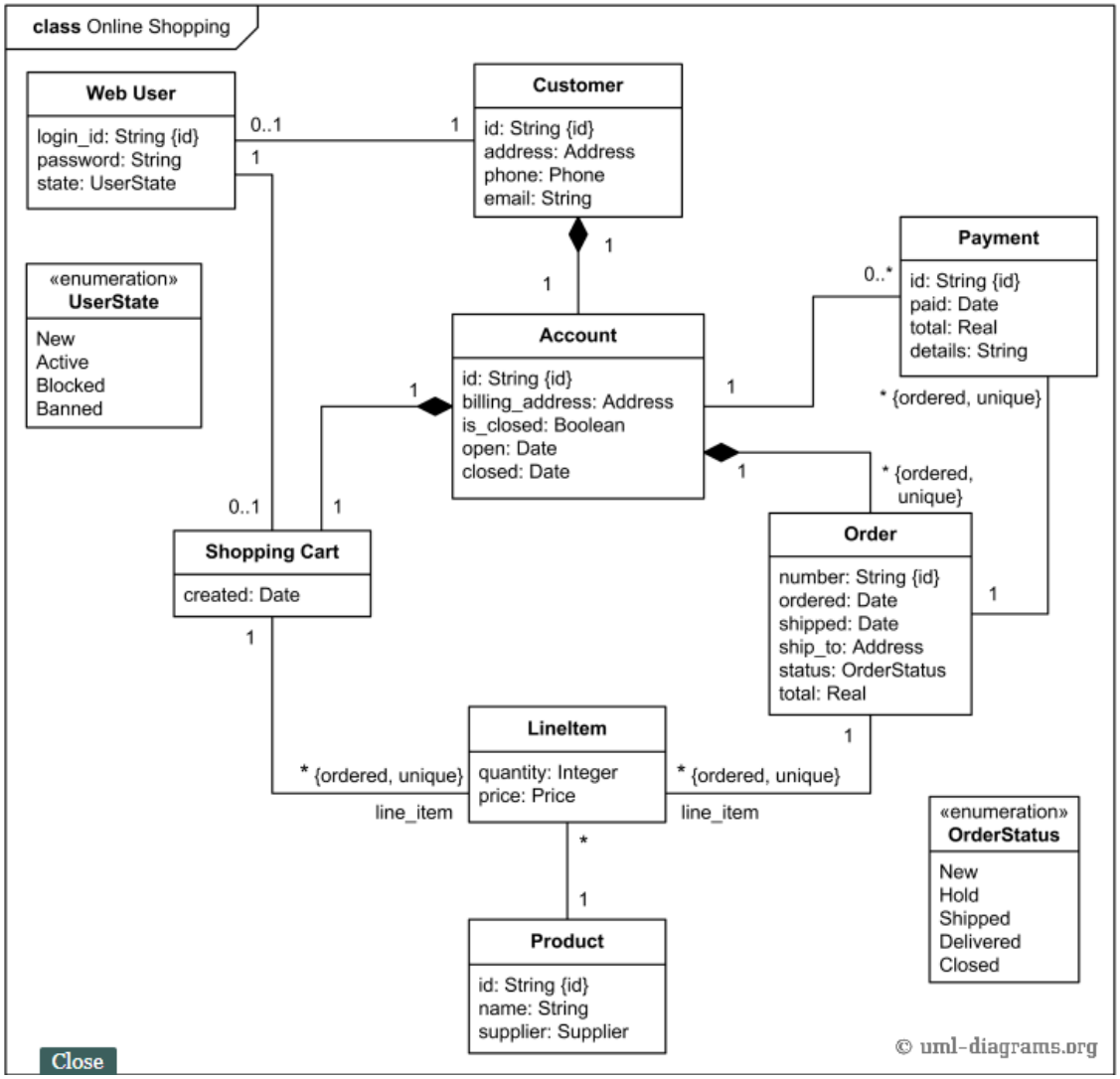


Figure A.9: Class Diagram For Online Shopping Application

A.4.3 Sequence Diagram

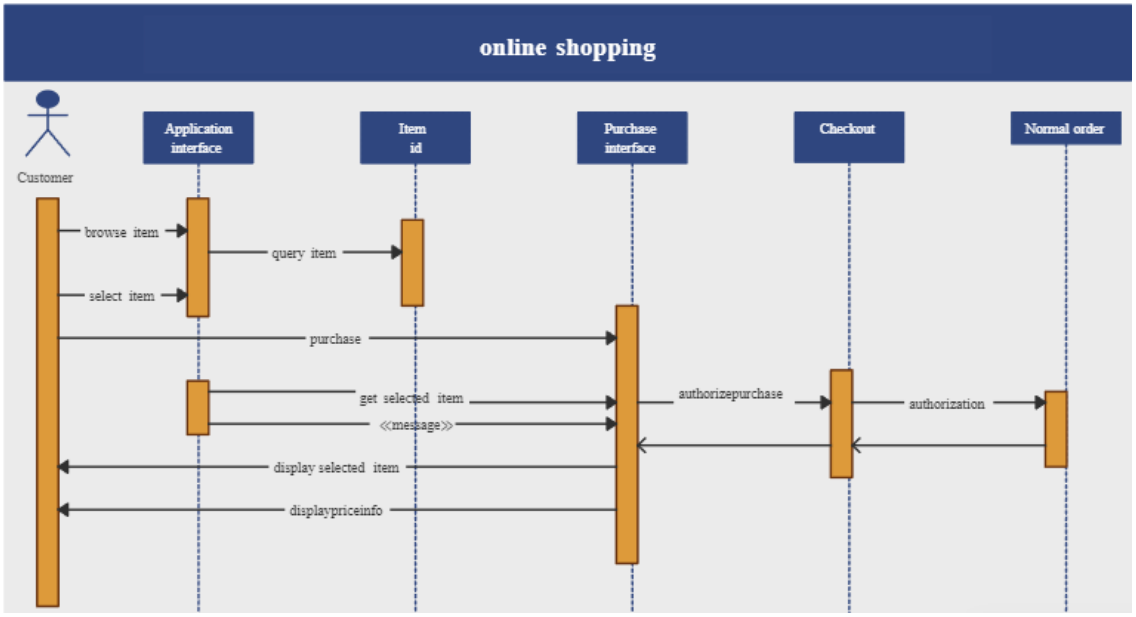


Figure A.10: Sequence Diagram For Online Shopping Application

A.4.4 State Transition Diagram

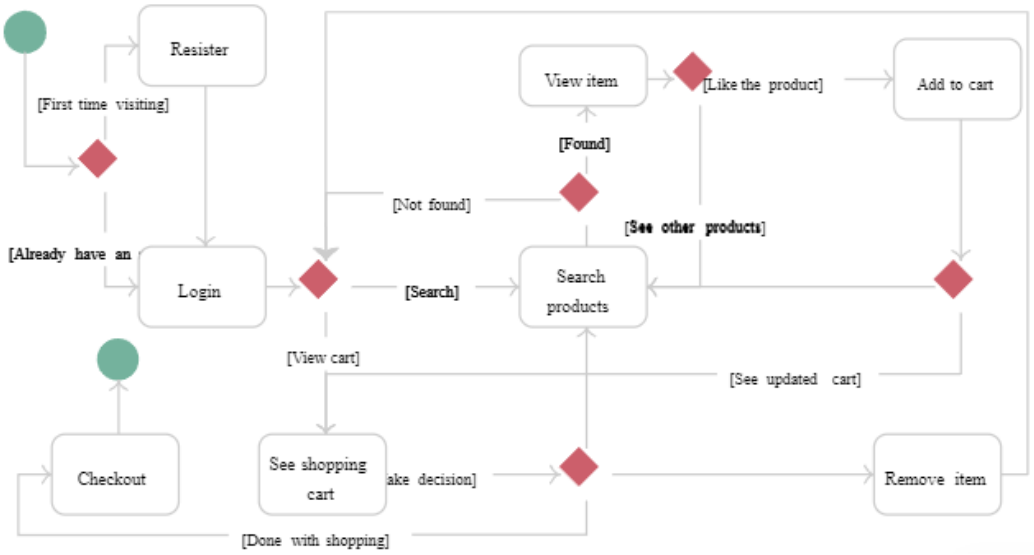


Figure A.11: State Transition Diagram For Online Shopping Application

A.4.5 Data Flow Diagram

Data Flow Diagram

Data flow diagram symbols, symbol names, and examples

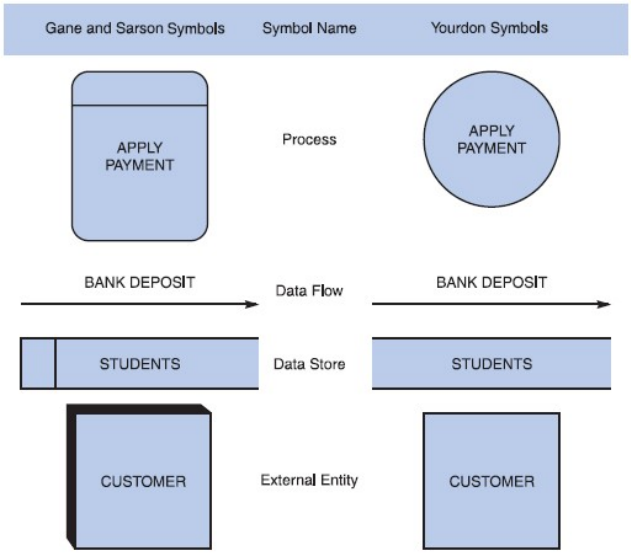


Figure B-5 Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Guidelines for Drawing DFDs

Step 1: Draw a Context Diagram: The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system's boundaries and scope. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

Example

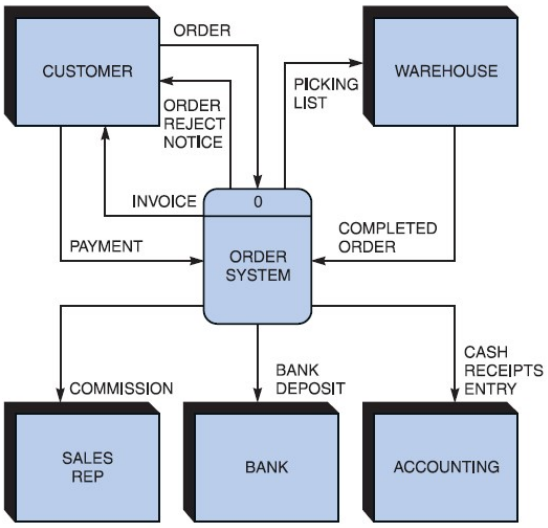


Figure B-6 Context diagram DFD for an order system.

Step 2: Draw a Diagram 0 DFD: To show the detail inside the black box, you create DFD diagram 0. **Diagram 0** zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.

Example

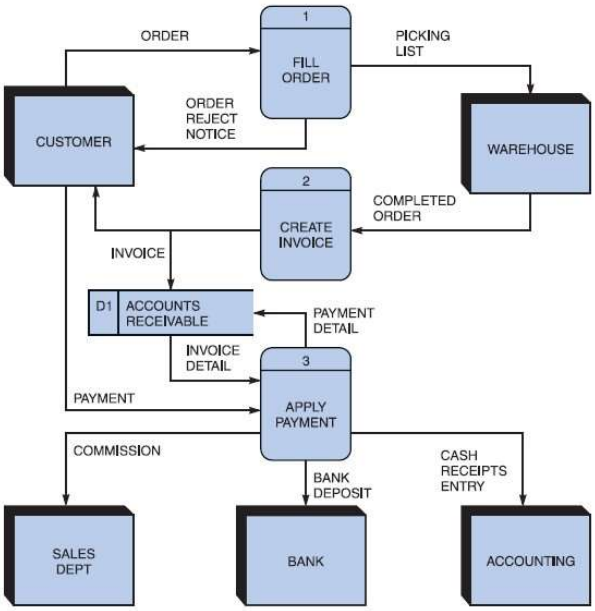


Figure B-7 Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams:

To create lower-level diagrams, you must use leveling and balancing techniques. **Leveling** is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified.

Leveling Example

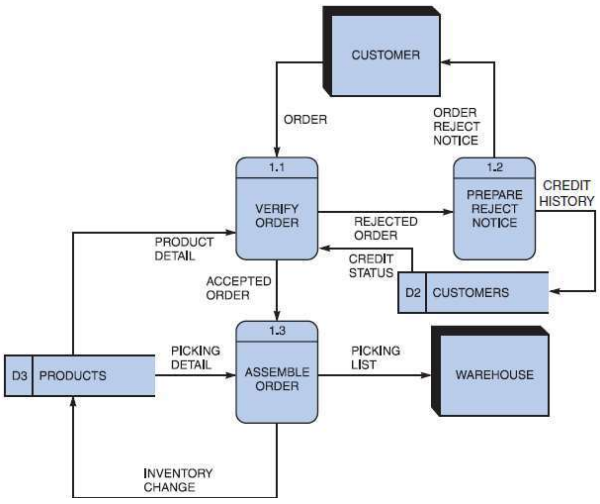


Figure B-8 Diagram 1 DFD shows details of the FILL ORDER process in the order system.